

João Pedro Coelho Encarnação

Modelo Articulado de uma Árvore Brônquica



Universidade do Algarve
Instituto Superior de Engenharia

2025/2026

João Pedro Coelho Encarnação

Modelo Articulado de uma Árvore Brônquica

Relatório de Projeto Final de Licenciatura
Licenciatura em Engenharia de Eletrotecnia e Computadores

Trabalho efetuado sob orientação do:
Professor Jorge Semião



Universidade do Algarve
Instituto Superior de Engenharia

2025/2026

Dedicatória

A preencher na versão final.

Agradecimentos

A preencher na versão final.

Resumo

O presente relatório descreve o desenvolvimento de um modelo articulado de uma árvore brônquica para apoio à simulação e treino de videobroncofibroscopia. O trabalho parte da necessidade de criar uma plataforma física de treino que permita representar, de forma controlada e realista, movimentos internos observáveis durante o procedimento, combinando técnicas como:

- impressão 3D;
- atuação mecânica;
- controlo eletrónico;
- software embarcado;
- interface homem-máquina.

A solução foi organizada em duas secções:

- Modelo dinâmico → constituído pela árvore brônquica impressa em 3D, pelos manipuladores delta, pelos servo-motores e pelo microcontrolador ESP32-S3;
- HMI → alojada num Raspberry Pi 4B, responsável por permitir ao utilizador controlar o sistema de forma simples e intuitiva, sem necessidade de memorizar comandos internos ou valores.

No modelo dinâmico foram implementados alguns perfis de movimento tais como:

- Respiração;
- Batimento cardíaco;
- Tosse.

Toda a implementação recorreu a estruturas de trajetórias, máquinas de estados, comunicação série e Equações de Cinemática Inversa para converter posições desejadas nos ângulos dos manipuladores. A interface gráfica por sua vez, permite selecionar modos de funcionamento, bem como estados de funcionamento (iniciar ou interromper o ensaio), outras funcionalidades como: estabelecer e acompanhar a comunicação série com o ESP32 e

preparar os parâmetros de movimento também se encontram implementadas, dando uma grande quantidade de ferramentas para os operadores. Durante todo o desenvolvimento foram validados, testados e modificados os varios blocos fundamentais dos sistemas já mencionados em ambos, modelo dinamico e HMI. Identificando-se no final as principais limitações do protótipo atual e possíveis sugeseões de melhorias a considerar no futuro.

Abstract

Índice Geral

	Página
Lista de Abreviaturas e Termos	xi
Lista de Figuras	xiii
Lista de Tabelas	xv
1 Introdução	1
1.1 Enquadramento	1
1.2 Motivação	2
1.3 Objetivos	2
1.4 Metodologia	3
1.5 Organização do relatório	3
2 Estado da Arte	4
2.1 Broncoscopia e treino por simulação	4
2.2 Fidelidade dos simuladores	4
2.3 Simuladores de broncoscopia	5
2.4 Impressão 3D aplicada à simulação médica	5
2.5 Movimentos observáveis durante a broncoscopia	6
2.5.1 Movimento respiratório	7
2.5.2 Movimento cardíaco	7
2.5.3 Tosse e perturbações rápidas	7
2.5.4 Interação com o broncoscópio	8
2.5.5 Patologias e deformações locais	8
2.6 Síntese crítica	8
3 Desenvolvimento do Hardware	10
3.1 Requisitos do sistema	10
3.2 Modelo físico da árvore brônquica	11
3.2.1 O modelo STL da árvore brônquica	11
3.2.2 Programas utilizados no tratamento do modelo	11

3.2.3	Materiais e problemas de impressão	12
3.3	Seleção do sistema de atuação	15
3.4	Movimentos representados no protótipo	16
3.5	Eletrônica de controlo e alimentação	17
3.6	Integração mecânica e segurança	18
4	Desenvolvimentos do Software e Configurações	21
4.1	Arquitetura geral do software	21
4.2	Ambiente de desenvolvimento	22
4.3	Estruturas de dados	23
4.4	Perfis de movimento	24
4.4.1	Geração de trajetórias na HMI (Python/Raspberry Pi) . . .	24
4.4.2	Execução dos perfis no modelo dinâmico (C++/ESP32) . . .	28
4.5	Máquina de estados de execução dos movimentos	30
4.6	Cinemática inversa do manipulador delta	35
4.7	Comunicação série e comandos	39
4.8	Interface HMI no Raspberry Pi	43
4.9	Preparação e sincronização da tosse	44
4.10	Depuração e validação interna	45
5	Testes e Análise de Resultados	47
	Nota sobre o estado atual do capítulo	47
5.1	Plano de testes	47
5.2	Testes do modelo físico	47
5.3	Testes dos atuadores e da estrutura delta	48
5.4	Testes de alimentação	48
5.5	Testes de software	48
5.6	Testes de comunicação série	49
5.7	Testes de integração dos movimentos	49
5.8	Critérios de aceitação	49
5.9	Informação experimental em falta	50
6	Conclusão	52
6.1	Síntese do trabalho	52
6.2	Principais contributos	52
6.3	Limitações	53
6.4	Trabalho futuro	53
	Bibliografia	55
A	Anexos	57
A.1	Anexo A: Desenhos técnicos e modelo 3D	57

A.2	Anexo B: Lista de materiais	57
A.3	Anexo C: Datasheets	57
A.4	Anexo D: Código e configurações	57
A.5	Anexo E: Protocolos de teste	57

Lista de Abreviaturas e Termos

ESP32-S3 Microcontrolador usado para executar o firmware, interpretar comandos e gerar os sinais de controlo.

GPIO *General Purpose Input/Output*; pinos digitais usados para ligação a periféricos.

PWM *Pulse Width Modulation*; técnica usada para controlar os servos através da largura de pulso.

UART *Universal Asynchronous Receiver/Transmitter*; interface de comunicação série usada entre o Raspberry Pi e o ESP32-S3.

HMI *Human Machine Interface*; interface homem-máquina usada para permitir ao utilizador controlar o modelo dinâmico através do Raspberry Pi.

FSM *Finite State Machine*; máquina de estados finitos usada para organizar a lógica de movimento, leitura série e interpretação de comandos.

IK *Inverse Kinematics*; cinemática inversa, usada para converter coordenadas cartesianas em ângulos dos servos.

LiPo Bateria de polímero de lítio.

MG90S Servo motor de engrenagens metálicas com um grau de liberdade de aproximadamente 180°.

PLA Filamento de impressão 3D de ácido polilático, usado sobretudo pela facilidade de impressão e rigidez.

PEBA Filamento de impressão 3D de polieter-bloco-amida, elastómero termoplástico flexível e leve.

TPU Filamento de impressão 3D de poliuretano termoplástico, usado em peças flexíveis e elásticas.

TPE Filamento de impressão 3D de elastómero termoplástico, família de materiais com comportamento elástico semelhante a borracha.

TPC Filamento de impressão 3D de copoliéster termoplástico, material flexível usado em peças elásticas e funcionais.

UPS *Uninterruptible Power Supply*; sistema de alimentação que mantém o equipamento ligado durante falhas ou interrupções temporárias da alimentação principal.

HAT *Hardware Attached on Top*; placa de expansão para Raspberry Pi, acoplada aos pinos GPIO para adicionar ferramentas, periféricos ou capacidade de processamento.

Lista de Figuras

3.1	Testes de impressão da árvore brônquica com TPU Morplex 95A, com suporte em PLA.	13
3.2	Servo motor com suporte impresso em 3D para validação do acoplamento mecânico com a estrutura do manipulador delta, março de 2026.	16
3.3	Caixa impressa em PLA branco que aloja o Raspberry Pi 4B e a fonte de alimentação comutada, junho de 2026.	18
3.4	Estrutura mecânica de um manipulador delta em PLA preto, antes da instalação dos servos, maio de 2026. A base circular e os três braços articulados estão já montados.	19
3.5	Os dois manipuladores delta montados com os servos Waveshare instalados, prontos para calibração de firmware, junho de 2026. Os cabos dos servos são visíveis mas ainda sem organização final de cablagem.	19
3.6	Esquema consolidado de alimentação, comunicação e comando dos dois manipuladores delta.	20
4.1	Estrutura global das máquinas de estados e blocos de controlo.	22
4.2	Ambiente de desenvolvimento de software e eletrónica: portátil com PlatformIO, osciloscópio RIGOL, ESP32 em breadboard e tablet com diagramas de apoio, maio de 2026.	23
4.3	Relação entre as estruturas de dados principais do firmware.	23
4.4	Trajectoria calculada para o movimento de batimento cardíaco, com representação 2D e 3D dos pontos de passagem.	26
4.5	Trajectoria calculada para o movimento respiratório, com representação 2D no plano ZX e visualização 3D.	27
4.6	Trajectoria calculada para a vibração associada à tosse, com representação 2D e 3D dos pontos de passagem.	29
4.7	Esquema da FSM_Motion_Update.	33
4.8	Representação lateral das variáveis usadas na cinemática inversa do manipulador delta.	36
4.9	Relação linear de referência entre o ângulo calculado pela cinemática inversa e o pulso PWM aplicado aos servos nos dois manipuladores.	38

4.10	FSM.Serial_Read responsável pela leitura e validação dos dados recebidos por comunicação série.	41
4.11	FSM.Serial_Command responsável pela interpretação dos comandos recebidos por comunicação série.	42
5.1	Sessão de observação preliminar com profissional de saúde: broncoscópio introduzido no modelo de árvore brônquica impresso em 3D enquanto o sistema de controlo estava ativo, junho de 2026.	51

Lista de Tabelas

3.1	Movimentos considerados para a primeira versão do protótipo.	17
3.2	Componentes principais previstos para o sistema.	20
4.1	Parâmetros ajustáveis usados no cálculo dos movimentos na HMI.	28
4.2	Pontos calculados para os movimentos usados nos testes da interface. . . .	32
4.3	Comandos simples disponíveis na comunicação série.	39
4.4	Subcomandos do menu de leitura, teste e depuração.	40
4.5	Subcomandos de seleção do modo fisiológico.	40

Capítulo 1

Introdução

Falta reescrever a apresentação/preparação inicial deste capítulo aqui Deve ser algo fácil de perceber e de "digerir".

1.1 Enquadramento

A broncoscopia flexível é um procedimento essencial em Pneumologia, Anestesiologia, Medicina Intensiva e Cirurgia Torácica, cuja execução exige coordenação psicomotora, reconhecimento anatómico, capacidade de navegação endoscópica e tomada de decisão num ambiente clínico variável. Por esse motivo, a simulação tem sido proposta como uma etapa relevante, no treino de novos médicos na área, para suavizar a curva de aprendizagem, permitindo treinos repetidos e aumentar a segurança antes da prática em doentes reais (será que vale apenas falar já aqui sobre as opções atuais dos vários treinos de simuladores??) [1–5].

O projeto apresentado neste relatório teve origem numa proposta de trabalho elaborada pela equipa de Pneumologia do Centro Hospitalar Universitário do Algarve [6], que identificou a necessidade de um modelo de simulação de árvore brônquica, impresso em 3D e de baixo custo, para treino de broncoscopia flexível. A partir dessa proposta, o objetivo deste trabalho passou a ser sobre o desenvolvimento de um modelo articulado de uma árvore brônquica, capaz de reproduzir, de forma legítima, os movimentos observáveis durante as broncoscopias, partindo da utilidade já reconhecida dos modelos físicos impressos em 3D e acrescentando-lhes uma componente dinâmica, de modo a aproximar a experiência de treino das condições reais de observação endoscópica [7, 8].

O projeto foi dividido em duas secções complementares:

- O Modelo dinâmico, que inclui todo o sistema de controlo e atuação motora (divididos em vários manipuladores em Delta) e o próprio modelo 3D da árvore brônquica;
- A HMI, ou interface homem-máquina, alojada num Raspberry Pi 4B com um monitor touchscreen, cuja sua função é permitir que o utilizador consiga facilmente

controlar o modelo dinâmico de forma intuitiva e natural, podendo selecionando os varios modos de funcionamento, referentes aos movimentos presentes no modelo, definindo parâmetros de funcionamentos e ordens, bem como os calculos e geração de coordenadas para os movimentos pretendidos, todas estas funcionalidades encontram-se associadas a varios botões e menus intuitivos, retirando assim a necessidade do usuario de decorar todos os comandos da comunicação série e dados de funcionamento.

1.2 Motivação

Existem varios simuladores comerciais de broncoscopia, contudo estes facilmente podem apresentar custos monetários extremamente elevados, o que limita a sua disponibilidade e acessibilidade em contextos de ensino e treino, razão pela qual a impressão 3D surge como uma forte alternativa, ao permitir criar modelos anatómicos acessíveis, personalizáveis e adaptáveis a diferentes níveis de dificuldade [7, 8]. Contudo, muitos destes modelos físicos permanecem como modelos estáticos, falhando na imersão da representação de fenómenos como a deslocação e respiratória das vias aéreas, variação de calibre dos brônquios, pequenas oscilações induzidas pelo coração e ou as perturbações rápidas associadas à tosse, por resposta do corpo humano à inserção de um corpo externo nos brônquios[9–12].

A motivação central deste trabalho tornou-se, portanto, na capacidade de combinar a acessibilidade e versatilidade dos modelos físicos impressos em 3D com a possibilidade implementação de movimentos anatomicos legítimos e comprovar assim a fiabilidade de um simulador dinamico realista, algo inovador comparado aos simuladores estaticos comuns.

1.3 Objetivos

Este projeto pretendia investigar, desenvolver, construir e documentar um protótipo articulado de uma árvore brônquica para treino de videobroncofibroscopia.

Os seus objetivos específicos foram:

- estudar requisitos anatómicos, mecânicos e pedagógicos para os simuladores de broncoscopia;
- desenvolver um plano e uma estrutura física compatível com impressão 3D e atuação mecânica;
- integrar sensores ou atuadores, e todo um sistema de eletrónica de controlo flexível, para os varios cenários de treinos;

- implementar firmware e configurações, pré feitas, de controlo para os diferentes perfis de movimento;
- desenvolver uma interface, HMI, para um controlo simples e intuitivo do modelo;
- definir uma estratégia de testes para avaliação da usabilidade, repetibilidade, robustez dos sistemas e verificação do sincronismo dos movimentos em termos de tempo e espaço percorrido pelos manipuladores robóticos;
- documentar as decisões de projeto, limitações e possíveis melhorias futuras.

1.4 Metodologia

A metodologia do trabalho combina revisão bibliográfica, definição de requisitos, desenvolvimento iterativo de hardware, desenvolvimento de **software Embutido** e preparação de testes funcionais, **não sei o que escrever mais aqui...**

1.5 Organização do relatório

O relatório está organizado em seis capítulos:

- Capítulo 1: Apresenta o enquadramento, a motivação e os objetivos;
- Capítulo 2: Revisão do estado da arte, incluindo treino por simulação, impressão 3D e movimentos observáveis durante a broncoscopia;
- Capítulo 3: Descreve o desenvolvimento do hardware;
- Capítulo 4: Apresenta estrutura de software, configurações internas, máquinas de estados e todo o sistema por detrás da HMI;
- Capítulo 5: Reúne o plano de testes e identifica a informação experimental ainda em falta;
- Capítulo 6: Sintetiza as conclusões e melhorias futuras.

Capítulo 2

Estado da Arte

Falta reescrever a apresentação/preparação inicial deste capítulo aqui Deve ser algo fácil de perceber e de "digerir".

2.1 Broncoscopia e treino por simulação

A broncoscopia flexível é um procedimento médico que permite observar diretamente a árvore traqueobrônquica, recolher amostras, auxiliar diagnósticos e apoiar intervenções terapêuticas. A aprendizagem deste procedimento exige coordenação entre visão endoscópica, manipulação fina do broncoscópio e interpretação anatômica. A simulação promotorizada é, por isso, uma ferramenta relevante para repetir tarefas, treinar orientação espacial e reduzir a dependência inicial de treino em doentes reais [1–4].

As revisões e estudos de treino em broncoscopia apontam para ganhos na aquisição de competências quando os simuladores são integrados de forma estruturada no ensino. O valor pedagógico de um simulador depende não só do realismo visual, mas também da relação entre o exercício proposto, os objetivos de aprendizagem e a forma como o desempenho é avaliado [2, 3, 5].

2.2 Fidelidade dos simuladores

A fidelidade de um simulador pode ser entendida em várias dimensões:

- Fidelidade anatômica → refere-se à correspondência entre a geometria do modelo e a anatomia real, como à forma, proporção e organização espacial das vias aéreas;
- Fidelidade mecânica → refere-se à resistência ao avanço, flexibilidade e resposta ao contacto;
- Fidelidade funcional → refere-se à capacidade de reproduzir movimentos, variações de calibre e eventos transitórios como a tosse.

Na broncoscopia, um modelo estático pode ser útil para situações de treinos iniciais de navegação e reconhecimento anatômico. No entanto, durante um procedimento real, o operador encontra-se num ambiente vivo e dinâmico, em constante movimento. A respiração, os batimentos cardíacos, a tosse, a sedação e a própria interação do broncoscópio com as paredes internas exercem comportamentos e reações que modificam continuamente o ambiente e a sensação de controlo. Esta diferença cria oportunidade para os simuladores físicos de baixo custo que incorporem movimentos e fisiologia de forma controlada [1, 4, 8].

2.3 Simuladores de broncoscopia

Os atuais simuladores de broncoscopia abordam varios tipos de sistemas tais como, treinos em ambientes puramente virtuais, em manequins comerciais, modelos de bancada e mais recentemente em modelos impressos em 3D: Os sistemas virtuais permitem métricas automáticas e cenários variáveis, mas podem ser dispendiosos e não conseguem introduzir a a sensação de limites e contacto, visto que é também uma metrica muito importante nestes treinos, entretanto os modelos físicos são mais simples e acessíveis, embora muitas vezes dependam de avaliação externa e tenham menor capacidade de reproduzir fenómenos dinâmicos (estou com dificuldades em expandir e explicar certos conceitos). Dando assim abertura para modelos impressos em 3d, para conquistar os inconvenientes dos outros modelos. (Preciso de ajuda!)

2.4 Impressão 3D aplicada à simulação médica

A impressão 3D é um tecnologia de produção de objetos tridimensionais, realizado através da adição de material camada a camada, podendo diversificar bastante no metodo ao qual é adicionando/aglomerado os materias de forma a gerar a forma pretendida (FSM, SLS, SLA). não sei se explique por alto os processos destas tecnologias Com o passar dos anos e amadurecimento da tecnologia e materiais disponiveis, este metodo de produção veio proporcionar uma oportunidade inovadora no que trata de produção e criação de estruturas e geometriias complexas a um baixo custo e fração do tempo em relação aos outros metodos de produção, afetado todas as as areas cientificas e industriais, inclusive a medicina, permitindo a produção de protótipos anatômicos com custo relativamente baixo, corrigir e ajustar dimensões em pouco tempo e testar materiais inovadores que com outros metodos era impensavel, tanto por conta do custo de produção ou devido á complexidade dos processos.

No caso dos simuladores de árvores brônquicas, s modelos impressos em 3D ganharam uma grande importância por permitirem transformar dados anatômicos (por vezes através de Scans 3d) em geometrias físicas e, em broncoscopia, esta abordagem pode repre-

sentar a árvore traqueobrônquica com boa correspondência espacial e permitir a criação de simuladores específicos para treino de navegação [7, 8], ainda que a sua principal limitação, quando usados de forma isolada, seja a ausência de movimento e de resposta fisiológica. Contudo a ideia de impressão destes modelos apresenta desafios e detalhes muito importantes, tais como a escolha do material, pois facilmente influencia o grau de dificuldade de impressão, a flexibilidade, a resistência a sensação ao toque e a disponibilidade de cor. Materiais rígidos, como PLA, podem servir para validar forma e montagem, enquanto materiais flexíveis, como TPU, TPE ou PEBA, podem aproximar-se melhor á resposta mecânica, ainda que aumentem consideravelmente a dificuldade de impressão e calibração. Assim, a impressão 3D deve ser entendida também como uma plataforma de iteração, não apenas como uma técnica de fabrico final.

2.5 Movimentos observáveis durante a broncoscopia

A árvore traqueobrônquica não se comporta como uma estrutura fixa durante uma broncoscopia. Mesmo quando o broncoscópio se mantém praticamente parado, a imagem observada pode apresentar deslocações lentas, alterações de orientação, variações do lúmen e perturbações rápidas. Para o operador, estes movimentos não aparecem isolados:

- A respiração cria uma componente periódica dominante;
- O batimento cardíaco acrescenta pequenas oscilações locais;
- A tosse introduz eventos abruptos;
- O contacto do broncoscópio com a parede pode modificar temporariamente a posição ou a forma do segmento observado.

Do ponto de vista de um simulador físico, estes fenómenos não precisam de ser reproduzidos com toda a complexidade fisiológica para terem valor pedagógico, mesmo que acabem sempre por apresentar um maior realismo. O mais importante é que o utilizador deixe de observar um modelo completamente estático e passe a treinar num ambiente com movimento por vezes imperscrutável e suficientemente coerente com o que é visto durante o procedimento real. Por isso, os movimentos descritos de seguida foram analisados quanto à sua relevância visual, dificuldade de implementação e utilidade, seguindo a categorização e os intervalos de frequência e amplitude propostos num documento de trabalho elaborado especificamente para este projeto pela equipa de Pneumologia do Centro Hospitalar Universitário do Algarve [13], que serviu de base clínica direta às secções seguintes.

2.5.1 Movimento respiratório

O movimento respiratório é a componente mais frequente e previsível, ocorrendo durante o processo cíclico da inspiração e expiração, descrito como a deslocação dos pulmões com predominância craniocaudal, acompanhados por pequenas componentes laterais e por alterações locais da geometria das vias aéreas. A frequência respiratória normal situa-se aproximadamente entre 12 e 20 ciclos por minuto, embora possa variar com ansiedade, sedação, esforço respiratório, patologia e contexto clínico.

Analises mostram que a fase respiratória pode alterar a posição de estruturas pulmonares e influenciar modelos de fluxo nas vias aéreas [11, 12], existindo ainda, além da deslocação global, variações do calibre brônquico ao longo do ciclo respiratório, que trabalhos clássicos já descreviam ao demonstrar métodos para as alterações de calibre dos brônquios durante a respiração normal e o mecanismo das suas variações rítmicas [9, 10]. Para o protótipo desenvolvido, esta componente foi interpretada como um movimento lento e periódico, adequado para ser representado por trajetórias **exponenciais** suaves e repetíveis.

2.5.2 Movimento cardíaco

O movimento cardíaco introduz oscilações mais rápidas e de menor amplitude, com intervalos de tempo entre as ocorrências, cuja influência tende a ser mais perceptível em regiões próximas do coração, em particular junto ao brônquio principal esquerdo e estruturas adjacentes, ao contrário da respiração, que domina a posição global da árvore brônquica, o batimento cardíaco surge antes como uma vibração ou perturbação sobreposta.

Num modelo de treino, esta componente é útil para quebrar a sensação de movimento puramente sinusoidal e acrescentar uma perturbação de menor amplitude, pelo que, no contexto deste projeto, o batimento cardíaco não foi tratado como uma reconstrução anatómica exata da interação coração-pulmão, **mas antes como um perfil adicional de movimento, mais rápido e discreto, combinado com a respiração. Verificar ainda esta frase, não sei se está bem escrita.**

2.5.3 Tosse e perturbações rápidas

A tosse é um evento curto, abrupto e menos previsível, provocando durante uma broncoscopia, a deslocação súbita das vias aéreas, com tendência para a ocorrência de replicas adjacentes á deslocação principal, alteração temporária do campo visual, variação do fluxo de ar e perda momentânea de orientação. Para quem treina, este tipo de evento é importante porque obriga a recuperar a posição visual e a manter controlo do broncoscópio após uma perturbação.

No simulador, a tosse deve por isso ser representada como um evento transitório,

com duração reduzida e amplitude superior à do batimento cardíaco, podendo a sua ocorrência ser programada de forma aleatória dentro de limites definidos, o que permite criar cenários de treino mais realistas sem obrigar à reprodução completa da fisiologia da tosse.

2.5.4 Interação com o broncoscópio

A passagem do broncoscópio também influencia a geometria observada, dado que o contacto com as paredes pode causar deformações locais, resistência ao avanço, pequenas oscilações e alterações da orientação da imagem; esta componente é especialmente importante para o realismo tátil, porque aproxima o simulador da sensação física de navegação no interior das vias aéreas.

Apesar da sua relevância, esta interação mostra-se a mais difícil de implementar. Exige materiais flexíveis adequados, geometrias com espessura e resistência, calibradas e idealmente, sensores ou mecanismos capazes de responder ao contacto. Levando por esse mesmo motivo a uma grande complexidade face à tentativa de implementar este tipo de acontecimento num simulador.

2.5.5 Patologias e deformações locais

Estenoses, obstruções, variações anatómicas e alterações patológicas modificam a navegação e o aspeto interno das vias aéreas, sendo que a impressão 3D permite criar geometrias específicas para representar alguns destes cenários de forma estática, o que já pode ser útil para treino de reconhecimento anatómico e planeamento de procedimentos.

No entanto, a simulação ativa de estenose variável, colapso local ou deformação dinâmica do lúmen exige um conjunto complexo de atuadores distribuídos, materiais compatíveis e uma estratégia de controlo mais complexa.

2.6 Síntese crítica

Esta análise comprova que existem necessidades que nem sempre são satisfeitas em simultâneo. Por um lado, os simuladores devem ser suficientemente acessíveis para poderem ser usados em contexto académico e profissional e de treinos repetidos. Por outro, devem evitar uma representação demasiado estática das vias aéreas, porque a broncoscopia real ocorre num ambiente anatómico em constante movimento.

Os modelos impressos em 3D respondem bem à primeira necessidade, pois permitem criar geometrias anatómicas de baixo custo e iterar rapidamente o projeto. Contudo, quando usados isoladamente, tendem a limitar-se à fidelidade geométrica. A componente dinâmica, mesmo que simplificada, acrescenta valor ao treino, pois obrigam o utilizador a

lidar com deslocações por vezes imperscrutáveis e á perda parcial do enquadramento, devido perturbações temporárias.

Assim, o objetivo deste projeto encontra-se, em combinar uma árvore brônquica física, obtida por impressão 3D, com uma arquitetura de atuação capaz de reproduzir três fenómenos anatómicos principais: respiração, batimento cardíaco e tosse.

Estes movimentos foram escolhidos por serem constantemente observáveis durante o procedimento, por terem impacto visual direto e por serem os mais viáveis de implementar. Fenómenos mais complexos, como variação ativa do calibre brônquico, resistência ao contacto e deformações patológicas dinâmicas, foram conscientemente descartados, por exigirem mecanismos e materiais que iam além do objetivo definido para o projeto, bem como a probabilidade de prelongar ainda mais a duração de desenvolvimento.

Capítulo 3

Desenvolvimento do Hardware

Este capítulo descreve o percurso de desenvolvimento do hardware, desde as primeiras impressões de teste até à montagem final do sistema dos dois manipuladores delta, eletrónica de controlo e suportes para guardar os componentes de forma segura. O processo não foi linear: cada iteração revelou limitações que levaram a novas abordagens do design, materiais e arquitetura mecânica. O que se segue documenta esse caminho de escolhas intermédias, bem os problemas que as motivaram.

3.1 Requisitos do sistema

Como já referido o modelo deve permitir o treino de videobroncofibroscopia com um modelo físico de baixo custo, robusto, (lavável) e fácil de montar, tendo os requisitos principais sido agrupados em quatro eixos:

- → Realismo anatómico;
- → Facilidade de utilização;
- → Movimento fisiológico controlado;

No plano anatómico, o modelo deve representar a árvore brônquica com dimensões aproximadas, ângulos de ramificação e características compatíveis com dados obtidos por tomografia computadorizada, enquanto no plano mecânico deve permitir a introdução de movimentos de respiração, batimento cardíaco e tosse sem comprometer a estabilidade do conjunto. No plano pedagógico, deve ainda permitir a repetição de exercícios e a adaptação do grau de dificuldade e, no plano técnico, (manter a solução acessível, evitando materiais, atuadores ou processos de fabrico que tornem o sistema demasiado caro para o objetivo do projeto).

Do ponto de vista funcional, o sistema foi separado em dois blocos:

- 1º → Modelo Dinâmico, constituído pela árvore brônquica impressa em 3D, pelos manipuladores delta, pelos servo-motores e seus respetivos drivers e pelo mi-

crocontrolador ESP32-S3, executando fisicamente os movimentos e mantendo um comportamento previsível mesmo quando recebe comandos externos;

- 2º → HMI, suportada pelo Raspberry Pi 4B e pelo monitor touchscreen, apresentando ao utilizador comandos de alto nível, como iniciar, parar, escolher modo fisiológico ou ajustar parâmetros, ficando a tradução desses comandos e suas execuções a cargo do software, fazendo com que o sistema seja o mais simples e intuitivo;

Um dos requisitos mais importantes do projeto é a capacidade de reproduzir movimentos fisiológicos de forma controlada, com frequências e amplitudes compatíveis com os valores observados em humanos, tornando-se também o desafio mais importante deste projeto, pois ambos os sistemas têm que funcionar de forma paralela e sincronizada, sem comprometer (Não sei como descrever os tempos e sincronismo das operações e movimentos).

3.2 Modelo físico da árvore brônquica

3.2.1 O modelo STL da árvore brônquica

O ponto de partida do projeto foi o modelo tridimensional da árvore brônquica, obtido a partir de dados de uma Tomografia (scan 3D) que foi realizada e entregue pelo grupo de médicos encarregues do projeto, em formato de malha 3D (STL). Este tipo de ficheiro define apenas a geometria da superfície do objeto, sob a forma de uma malha de triângulos, sem qualquer volume ou espessura associada, pelo que o modelo obtido correspondia apenas à superfície interna da árvore brônquica, uma espécie de casca oca, sendo por isso necessário atribuir-lhe espessura manualmente antes da impressão 3D.

3.2.2 Programas utilizados no tratamento do modelo

Esta operação foi feita inicialmente no programa gratuito Tinkercad (da Autodesk), com o objetivo de realizar um teste rápido em PLA e confirmar a viabilidade geométrica da impressão. Depois de uma confirmação visual, passou-se para um programa mais indicado para este trabalho, o Fusion 360, um programa CAD da Autodesk destinado ao desenho 3D de sólidos, superfícies e malhas, que dispõe de uma ferramenta interna chamada “Forms”. Ao contrário do ambiente normal do Fusion 360, onde se trabalha maioritariamente com objetos sólidos e geometria bem definida, o Forms é exclusivamente indicado para trabalhar diretamente com scans 3D e as suas irregularidades geométricas, o que o tornava particularmente adequado a este caso.

No ambiente Forms, não se edita diretamente o ficheiro de malha; em vez disso, é criada uma aproximação à superfície do ficheiro através de uma camada virtual, sendo

apenas a essa camada que depois é possível atribuir um valor de espessura, criando assim um verdadeiro objeto 3D da árvore brônquica. Esta abordagem permitia editar em tempo real a geometria da camada e corrigir irregularidades existentes na malha, bem como descartar secções consideradas desnecessárias, como por exemplo as extremidades mais pequenas do scan 3D, que não tinham qualquer utilidade para o modelo devido à incompatibilidade entre a largura das vias nessas extremidades e a largura da sonda broncoscópica.

Contudo, observou-se rapidamente que este método era demorado e ineficiente. O ambiente Forms trabalha com formas primitivas do tipo T-Spline, como o cilindro sem base nem topo ou uma superfície do tipo folha, compostas por vértices, arestas e faces que podem ser deslocados, rodados ou escalados através da ferramenta Edit Form, moldando a forma até se aproximar da geometria pretendida. Para este modelo, o cilindro sem tampas foi a forma escolhida, por já se aproximar naturalmente do formato dos ramos da árvore brônquica. Não era, no entanto, possível aplicar esta aproximação ao modelo completo de uma só vez. Era necessário trabalhar ramo a ramo, ajustando individualmente um destes cilindros a cada ramificação através da ferramenta Edit Form, para depois ser cosido aos ramos adjacentes e ter a posição dos pontos novamente afinada, de forma a manter a fidelidade à geometria original do scan. Este processo, repetido ramo a ramo ao longo de toda a árvore brônquica e seguido de sucessivas costuras e correções, tornava o método muito demorado. Por esta razão, optou-se por um outro programa, o MeshMixer, também da Autodesk, direcionado exclusivamente para trabalhar e modificar diretamente ficheiros de malha 3D. Ao contrário do Forms, que edita apenas uma camada virtual de aproximação, o MeshMixer permite selecionar diretamente os triângulos da malha através de ferramentas de pincel e laço, corrigindo na fonte as imperfeições e cortando as secções desnecessárias. A ferramenta Inspector fecha automaticamente as malhas abertas e repara falhas de continuidade na superfície, enquanto o Offset permite atribuir espessura diretamente à malha, sem depender de uma camada intermédia como no Forms. Uma outra vantagem do MeshMixer em relação ao Forms está na forma como lida com sobreposições. Quando, com o aumento de espessura (scale-up), duas malhas ficam sobrepostas, a ferramenta Boolean Union do MeshMixer une-as automaticamente numa só peça, enquanto o Fusion 360 não consegue processar essas sobreposições e obriga a manter uma distância mínima entre elas, exigindo várias iterações e ajustes manuais.

3.2.3 Materiais e problemas de impressão

A seleção de materiais foi considerada uma decisão crítica ao longo de todo o projeto, tendo sido identificados filamentos flexíveis como PEBA, TPU, TPE, TPC e PP, bem como diferentes durezas comerciais, por exemplo 95A, 85A e 70A. Como o objetivo do modelo era aproximar-se o mais possível da realidade anatômica, era necessário utilizar



Figura 3.1: Testes de impressão da árvore brônquica com TPU Morplex 95A, com suporte em PLA.

um material suficientemente mole, sem que se soubesse à partida qual o grau de maciez adequado, pelo que foi necessário testar vários materiais, geometrias e durezas ao longo do projeto.

(podes inserir aqui uma tabela com a escala de durezas Shore dos filamentos flexíveis) Nos filamentos maleáveis, quanto menor a dureza do material, mais difícil se torna a sua impressão. Os filamentos mais duros alimentam-se com facilidade através do extrusor, enquanto os mais moles tendem a comprimir-se e a deformar-se nas engrenagens de alimentação em vez de serem corretamente empurrados, aumentando o risco de encravamento. A dureza mais comum no mercado é a 95A, existindo ainda assim empresas que produzem filamentos especializados com durezas tão baixas como 85A ou mesmo 70A, sendo este último especialmente problemático, por se deformar com facilidade durante a impressão e por exigir um extrusor direct-drive dedicado para não encravar. Por esta razão, optou-se inicialmente por testar um material mais comum e mais fácil de trabalhar, o TPU Morplex 95A, com suporte impresso em PLA, conforme documentado nos primeiros testes de impressão (figura 3.1).

Um outro desafio foi decidir como dispor o modelo para impressão em FDM, a única tecnologia disponível na universidade. Uma impressora SLA eliminaria alguns destes problemas de suporte, mas foi desde logo descartada:

- Vantagem do SLA: dispensaria suportes em várias zonas do modelo, simplificando a geometria de impressão;
- Desvantagem do SLA: equipamento com custo muito superior e dependente de equipamento adicional de pós-processamento, o que tornava a sua aquisição incomportável para o projeto.

Mantendo-se a impressão em FDM, foram equacionadas três opções para a dis-

posição do modelo:

1. **Impressão completa com suportes exteriores.** O modelo seria impresso de uma só vez, com suportes colocados do lado de fora para sustentar as zonas sem apoio inferior, que ficariam essencialmente a ser impressas “no ar”:
 - Vantagens: simplicidade de modificação da impressão e possibilidade de reaproveitar os próprios suportes como estrutura de fixação do modelo à base com os atuadores;
 - Desvantagens: tempo de impressão elevado, uma vez que todo o modelo seria impresso de uma só vez, obrigando a reimprimir a peça inteira em caso de falha, com desperdício considerável de filamento, tempo e energia, além de os suportes só poderem ser colocados no exterior do modelo, deixando sem sustentação zonas interiores igualmente sujeitas a grandes deformações, sobretudo em materiais mais moles.
2. **Separação e desdobramento das ramificações.** Esta opção passava por separar literalmente cada ramificação da malha num objeto independente e desdobrar a secção cilíndrica de cada ramo num plano, à semelhança do desdobramento de papel para construir um sólido, atribuindo-lhe espessura já na forma planificada e devolvendo-lhe a forma tridimensional apenas depois de impresso:
 - Vantagens: possibilidade de modificar facilmente cada ramo de forma individual e de integrar sensores ou atuadores diretamente no modelo, caso necessário;
 - Desvantagens: incerteza quanto à viabilidade do método, uma vez que, embora fosse possível desdobrar um ficheiro STL num plano com programas como o Blender, não havia garantia de que o mesmo funcionasse com um scan tão complexo e detalhado, podendo gerar erros ou geometrias inesperadas; a montagem após a impressão era também um risco, dado que os filamentos maleáveis são imprevisíveis e podem esticar ao serem retirados da base de impressão, criando folgas na união das peças e exigindo uma estratégia própria de costura entre secções, com uma probabilidade reduzida de sucesso ao longo de tantas ramificações.
3. **Molde negativo com vazamento de resina ou silicone.** Em vez de imprimir diretamente o modelo, esta opção previa a impressão de um molde negativo, utilizado depois para vaziar resina ou silicone e obter o positivo da árvore brônquica. Foi uma ideia forte durante grande parte do projeto, funcionando quase sempre como plano secundário:

- Vantagens: possibilidade de usar um material mais rígido no molde e um material extremamente mole no modelo positivo, bem como reutilizar o mesmo molde para produzir vários modelos sem o descartar;
- Desvantagens: impossibilidade de criar um único molde positivo para as partes interna e externa do modelo, uma vez que este é composto por duas secções ocas, interior e exterior, sendo no fundo apenas uma película fina com a forma da árvore brônquica, o que exigia unir três secções positivas (o interior e os dois lados do molde exterior); a orientação das ramificações em várias direções dificultava também a remoção do molde sem danificar o modelo, e a geometria complexa e extensa não dava garantias de que fosse possível injetar resina ou silicone de forma uniforme em todos os cantos, com risco de inconsistências na espessura das paredes.

Para as versões de teste das peças estruturais dos manipuladores delta utilizou-se PLA preto, por ser fácil de imprimir, rígido e acessível. Para a versão final da árvore brônquica, utilizada nos testes com profissionais de saúde, a impressão foi feita em filamento flexível numa impressora Bambu Lab de maior capacidade, conforme mostra a ??, mudança de equipamento e material que permitiu imprimir o modelo completo numa só peça, com melhor acabamento de superfície e menor fragilidade nas zonas de bifurcação.

3.3 Seleção do sistema de atuação

Foram consideradas duas abordagens para gerar movimento no modelo. A primeira consistia em distribuir vários atuadores ao longo da árvore brônquica, cada um com um movimento independente. Esta solução seria relativamente simples de programar para movimentos locais, mas tornaria o sistema mais difícil de gerir mecanicamente, aumentaria a quantidade de motores e exigiria uma estrutura de suporte complexa acoplada ao modelo.

A segunda abordagem, aprovada para o desenvolvimento, consiste na utilização de manipuladores robóticos do tipo delta. Esta opção permite controlar pontos da estrutura nos três eixos cartesianos, concentrando a complexidade mecânica em unidades de atuação repetíveis e, embora introduza maior complexidade matemática, devido ao cálculo da cinemática inversa, permite reproduzir vários movimentos com menos alterações físicas ao hardware.

Foram analisados dois tipos de manipuladores delta: o primeiro utiliza três atuadores e permite controlar a posição de um corpo em três eixos, X , Y e Z , enquanto o segundo utiliza seis motores, permitindo também controlar orientações adicionais. Para esta versão do projeto, foi considerado suficiente o controlo em três eixos, uma vez que os movimentos inicialmente acordados foram respiração, batimento cardíaco e tosse.

Esta escolha mecânica foi acompanhada, ainda no final de abril de 2026, pelo início do desenvolvimento do código de cinemática inversa associado a este tipo de manipulador, descrito em detalhe na section 4.6. A validação cruzada entre o comportamento físico dos braços montados em bancada e o cálculo implementado no firmware foi, aliás, uma constante ao longo de todo o desenvolvimento mecânico que se segue.

Antes da montagem final, foram realizados testes de acoplamento mecânico entre os servos e as peças impressas, utilizando suportes em PLA para confirmar as tolerâncias de encaixe e o posicionamento da patilha de cada servo em relação à base do manipulador (figura 3.2).

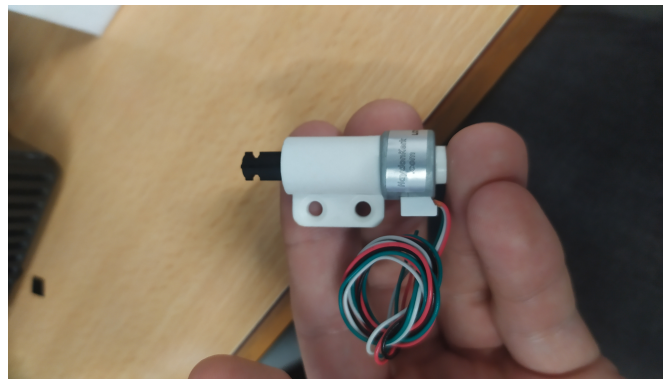


Figura 3.2: Servo motor com suporte impresso em 3D para validação do acoplamento mecânico com a estrutura do manipulador delta, março de 2026.

3.4 Movimentos representados no protótipo

O estado da arte identificou vários movimentos observáveis durante a broncoscopia. No entanto, para a primeira versão funcional do modelo, foram selecionados apenas os movimentos com melhor relação entre relevância clínica, viabilidade mecânica e simplicidade de controlo: respiração, batimento cardíaco e tosse. Os valores de frequência e amplitude usados nas secções seguintes, resumidos na tabela 3.1, foram retirados diretamente do documento de trabalho fornecido pela equipa de Pneumologia do CHUA [13].

A respiração foi definida como um movimento cíclico, com frequência aproximada de 12 a 20 ciclos por minuto e deslocamento predominantemente craniocaudal, devendo a implementação permitir pequenas irregularidades e alteração de amplitude, para que o movimento não pareça completamente artificial.

O batimento cardíaco foi associado principalmente ao brônquio principal esquerdo, devido à proximidade anatômica do coração, prevendo-se um movimento de baixa amplitude, cerca de 1 a 3 mm, com frequência aproximada de 60 a 120 batimentos por minuto; no protótipo, este movimento é tratado como uma oscilação sobreposta ao movimento respiratório.

A tosse foi caracterizada como um evento abrupto, irregular e de maior amplitude relativa e, em vez de ser apenas uma trajetória independente, deve alterar temporariamente o comportamento respiratório, aumentando a amplitude e introduzindo uma perturbação rápida. Movimentos como deformação ativa do lúmen, estenose localizada e resistência ao contacto com a sonda não foram incluídos nesta versão, por exigirem atuadores distribuídos e materiais com características mecânicas mais específicas.

Tabela 3.1: Movimentos considerados para a primeira versão do protótipo.

Movimento	Frequência	Amplitude pre- vista	Observações
Respiração	12–20 ciclos/min	5–10 mm	Cíclico, com pequenas irregularidades
Batimento cardíaco	60–120 bpm	1–3 mm	Oscilação de baixa amplitude
Tosse	Evento curto	5–25 mm	Abrupto, não linear e irregular

3.5 Eletrónica de controlo e alimentação

A arquitetura prevista separa a interface de utilizador e o cálculo de alto nível do controlo direto dos atuadores, cabendo ao Raspberry Pi 4B a responsabilidade pela HMI, pela monitorização e pelo envio de comandos ou posições, enquanto o ESP32-S3 atua como microcontrolador de tempo real, recebendo dados por comunicação série e convertendo-os em comandos para os servos.

O sistema de alimentação foi definido considerando pelo menos dois níveis: 5 V para o Raspberry Pi 4B e o ESP32-S3, e 5 V a 6 V para os servos. Durante a seleção dos atuadores também foi considerada a possibilidade de utilizar motores de passo, que exigiriam tensões superiores, por exemplo 8 V a 12 V, tendo sido estimados, para os servos, consumos nominais de cerca de 150 a 250 mA por unidade, com valores superiores em bloqueio.

Foi considerada a utilização de uma UPS/HAT para o Raspberry Pi, permitindo alimentação pelos pinos de 5 V e transição para bateria em caso de falha, tendo sido analisada a opção de um HAT de gestão de energia do tipo Suptronics X1209, por combinar alimentação, bateria e gestão de entrada no mesmo módulo. O ESP32, por sua vez, pode ser alimentado pelo Raspberry Pi através de USB-C, mantendo simultaneamente a ligação série necessária à comunicação.

Para acomodar o Raspberry Pi 4B e a fonte de alimentação de forma compacta e robusta, foi projetada e impressa uma caixa em PLA branco. Esta estrutura, visível na figura 3.3, agrupa o Raspberry Pi com a fonte de alimentação comutada numa única

unidade que pode ser posicionada independentemente do modelo dinâmico.

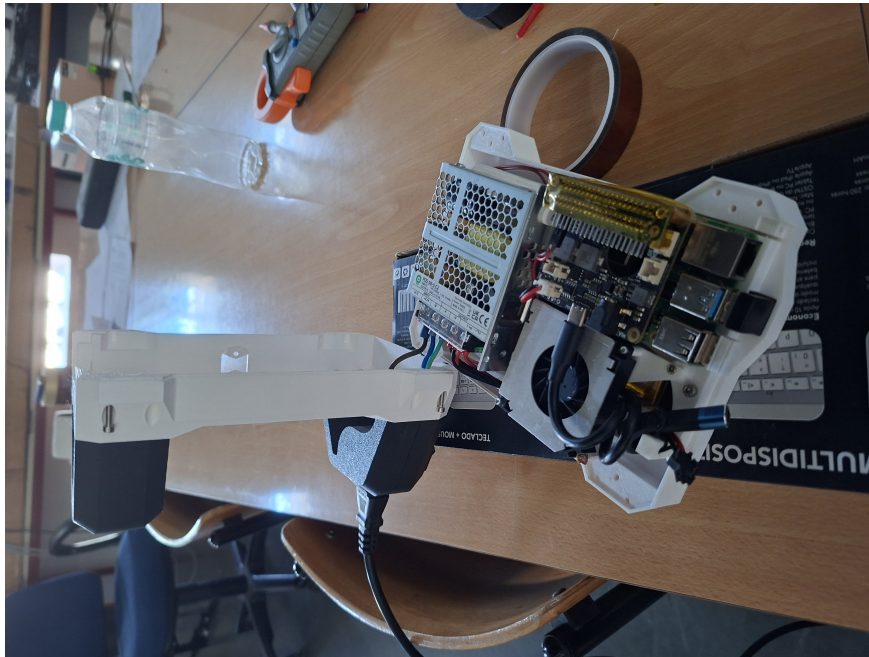


Figura 3.3: Caixa impressa em PLA branco que aloja o Raspberry Pi 4B e a fonte de alimentação comutada, junho de 2026.

3.6 Integração mecânica e segurança

A integração mecânica deve garantir que o movimento é transmitido à árvore brônquica sem esforços excessivos, folgas significativas ou interferência com a passagem do broncoscópio, devendo a estrutura permitir ainda acesso para manutenção, limpeza e substituição de peças. Do ponto de vista de segurança, os limites de curso do manipulador e os limites angulares dos servos devem ser definidos no firmware, evitando posições impossíveis ou capazes de danificar o modelo.

A fase de montagem começou pela impressão das peças estruturais em PLA preto, incluindo a base circular, os braços superiores, os elos de ligação e a plataforma móvel central. Numa primeira fase, a estrutura foi montada sem servos para verificar folgas, alinhamento das articulações e comportamento geométrico do conjunto (figura 3.4), etapa que permitiu identificar zonas onde era necessário rever tolerâncias e confirmar que os braços articulados rodavam sem bloqueio.

A instalação dos servos foi feita de forma gradual, com validação individual de cada canal, tendo os dois manipuladores sido testados, numa fase intermédia, com ligações diretas ao ESP32 sem PCB de interface, usando jumpers para confirmar o funcionamento elétrico básico antes de qualquer ligação definitiva. A figura 3.5 mostra os dois manipuladores montados com todos os servos instalados, prontos para a fase de calibração de firmware.



Figura 3.4: Estrutura mecânica de um manipulador delta em PLA preto, antes da instalação dos servos, maio de 2026. A base circular e os três braços articulados estão já montados.



Figura 3.5: Os dois manipuladores delta montados com os servos Waveshare instalados, prontos para calibração de firmware, junho de 2026. Os cabos dos servos são visíveis mas ainda sem organização final de cablagem.

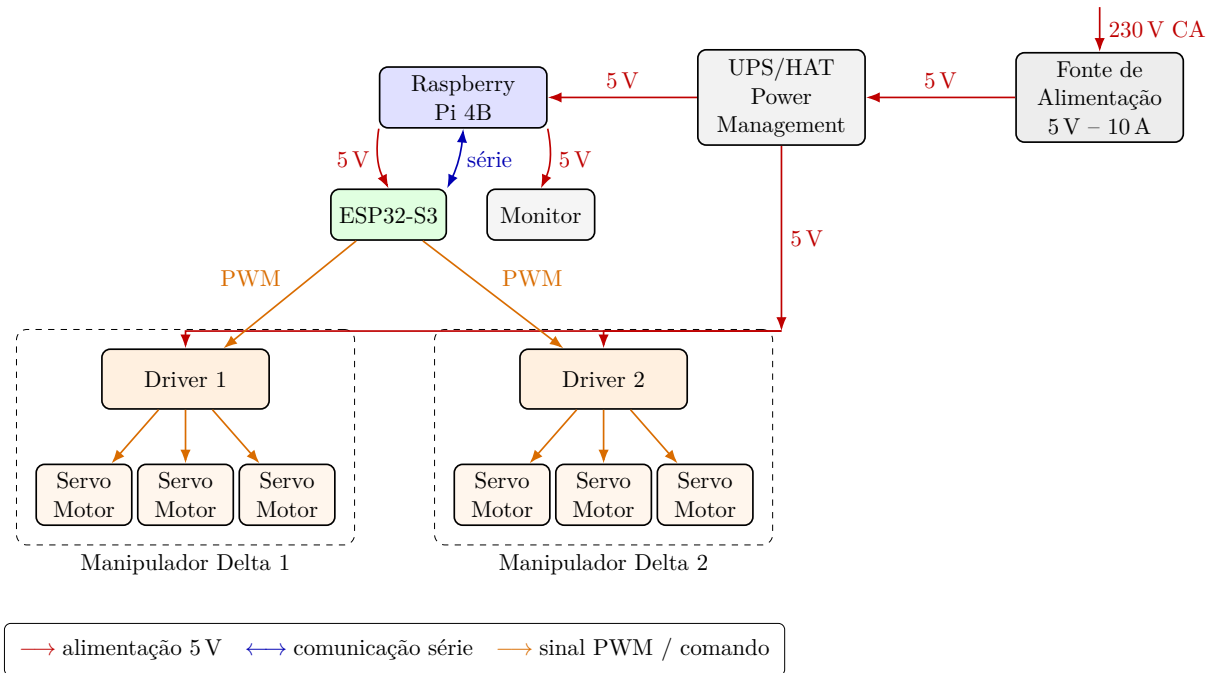


Figura 3.6: Esquema consolidado de alimentação, comunicação e comando dos dois manipuladores delta.

Tabela 3.2: Componentes principais previstos para o sistema.

Componente	Função	Observações
Raspberry Pi 4B	Interface e coordenação	Envia comandos ou posições para o ESP32
ESP32-S3	Controlo dos atuadores	Executa FSMs e gera comandos para servos
Servo-motores	Atuação mecânica	Seis unidades distribuídas pelos dois manipuladores delta
Driver Servo-motores	Interface de potência/comando	Cada driver comanda três servo-motores de um manipulador delta
UPS/HAT	Gestão de energia	Alimentação do Raspberry Pi e transição para bateria
Fonte de Alimentação 5 V-10 A	Alimentação principal	Recebe 230 V e fornece 5 V ao sistema
LiPo Battery 2100 mA 3,7 V	Alimentação de reserva	Bateria associada ao sistema UPS/HAT

Capítulo 4

Desenvolvimentos do Software e Configurações

O software foi desenvolvido em paralelo com o hardware, mas em camadas progressivas. Começou por testes elementares de comunicação série para confirmar que o ESP32 e o Raspberry Pi conseguiam trocar dados de forma fiável. A partir daí foram adicionados, um a um, os blocos de geração de trajetórias, de máquinas de estados, de cinemática inversa e de interface gráfica. Este capítulo descreve essa construção por partes, da arquitetura global até aos detalhes de calibração e sincronização, seguindo a ordem em que cada bloco foi desenvolvido e validado.

4.1 Arquitetura geral do software

O software foi organizado para separar a definição dos movimentos, o estado de execução, a comunicação série, a cinemática inversa, o acionamento dos servos e a interface HMI. Esta separação permite que os mesmos mecanismos de execução sejam usados para diferentes fenómenos fisiológicos, como respiração, batimento cardíaco e tosse, mantendo a interface de utilizador independente da lógica de baixo nível executada no ESP32-S3.

A arquitetura global, representada na figura 4.1, organiza o controlo em blocos independentes de HMI, comunicação, interpretação de comandos, atualização de movimento e cálculo cinemático, onde o Raspberry Pi executa a interface gráfica e envia comandos por comunicação série, enquanto o ESP32-S3 lê esses dados, interpreta comandos, atualiza estruturas de movimento, prepara eventos como a tosse e calcula as posições finais que serão convertidas pela cinemática inversa.

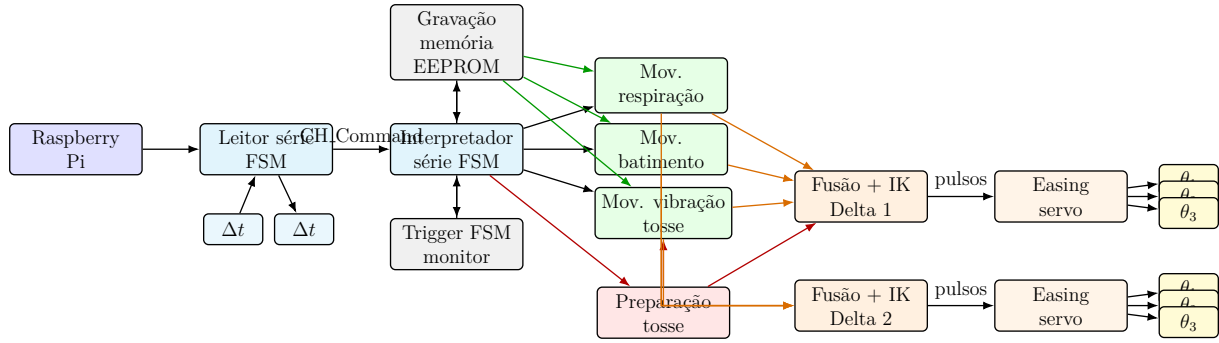


Figura 4.1: Estrutura global das máquinas de estados e blocos de controle.

4.2 Ambiente de desenvolvimento

O firmware foi desenvolvido em C++ para a plataforma ESP32, usando o ecossistema Arduino/PlatformIO, recorrendo à biblioteca `ESP32Servo` para o controlo dos servos e à biblioteca `Adafruit_NeoPixel` para a indicação visual de estados por LED RGB.

O ponto de partida deste firmware foi um exemplo de referência para manipuladores delta, disponível publicamente, adaptado no final de abril de 2026 à geometria e aos servos usados neste projeto. Depois de confirmado o funcionamento básico da cinemática e do acionamento dos servos, esse código de referência foi progressivamente separado do código próprio do projeto, o que permitiu reescrever cada bloco — estruturas de dados, geração de movimento, máquinas de estados — de forma independente, sem manter a organização original do exemplo.

O ESP32 comunica com o computador ou com o Raspberry Pi 4B por porta série a 115200 baud, tendo sido também usados, durante o desenvolvimento, scripts Python para leitura da porta série, depuração de mensagens e validação de comandos enviados pelo microcontrolador.

Antes da construção da HMI final, foram criados testes simples de comunicação série em Python. Estes testes permitiam abrir a porta série, controlar os sinais DTR/RTS, ler mensagens enviadas pelo ESP32 e introduzir comandos manualmente no terminal. Esta etapa foi importante para confirmar que o backbone de comunicação entre o computador/Raspberry Pi e o ESP32 estava funcional antes de se acrescentar uma interface gráfica. Em paralelo, foram usados códigos de cálculo de movimentos para validar a geração de pontos, curvas e gráficos que mais tarde passariam a ser integrados na interface. Estes scripts de comunicação e de cálculo foram desenvolvidos e validados de forma isolada, ainda durante maio de 2026, antes de qualquer integração com uma interface gráfica.

A figura 4.2 mostra o ambiente de trabalho típico desta fase: o portátil com o PlatformIO aberto a lado de um osciloscópio RIGOL utilizado para verificar os sinais PWM, o ESP32 montado em breadboard com jumpers para os servos, e uma tablet com os diagramas das máquinas de estados usados como referência durante o desenvolvimento.

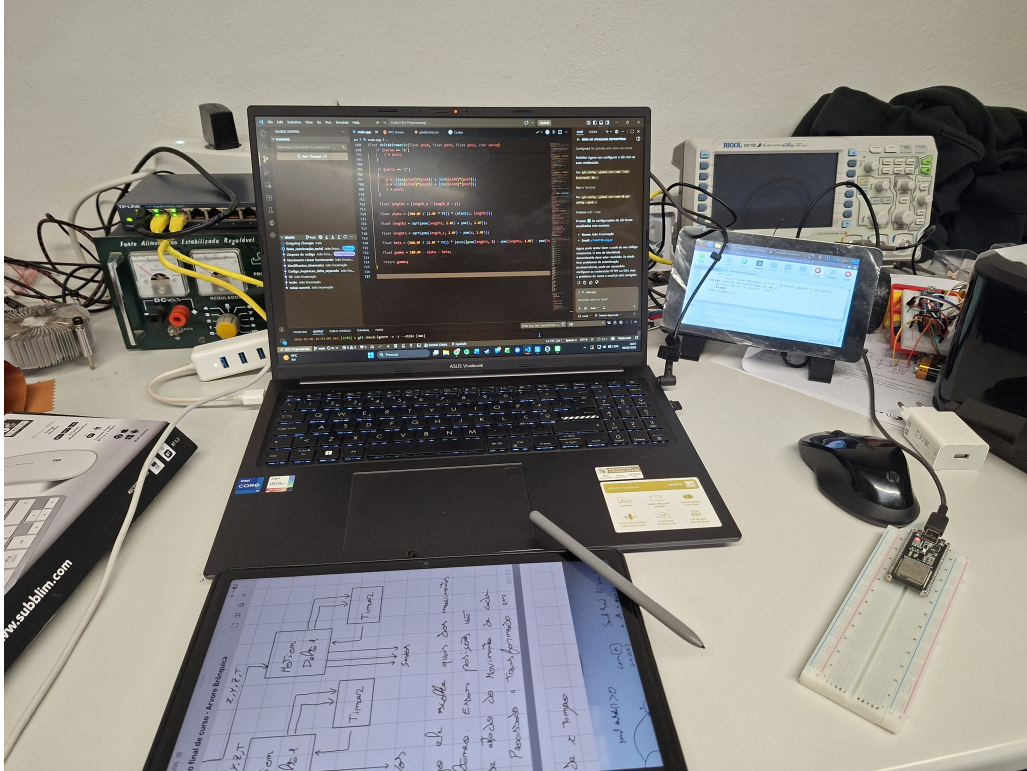


Figura 4.2: Ambiente de desenvolvimento de software e eletrônica: portátil com PlatformIO, osciloscópio RIGOL, ESP32 em breadboard e tablet com diagramas de apoio, maio de 2026.

4.3 Estruturas de dados

A estrutura `Manipulador_Config` concentra os parâmetros geométricos e de calibração, entre os quais se encontram os comprimentos $L1$, $L2$ e $L3$, os offsets do servo nos eixos X e Z , os limites angulares e os valores mínimos e máximos de pulso para cada servo.

A estrutura `MotionStorage` guarda a definição de uma trajetória, incluindo arrays de pontos X , Y e Z para cada movimento, número de pontos, período total, tempos de pausa no início e no fim, índice inicial e informação sobre se o movimento é bidirecional; já a estrutura `MotionInstance` representa o estado dinâmico de uma trajetória em execução, guardando ponteiros, flags, índices, direção, tempos decorridos e estado atual da FSM.

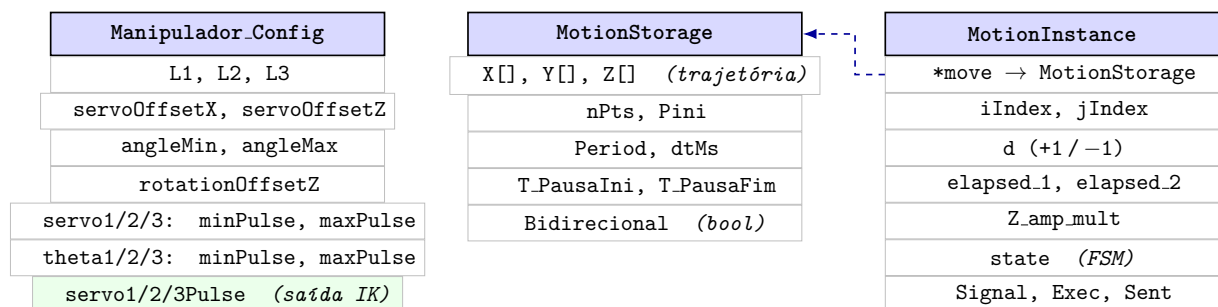


Figura 4.3: Relação entre as estruturas de dados principais do firmware.

4.4 Perfis de movimento

Foram definidos três perfis principais, procurando transformar a descrição qualitativa dos movimentos observáveis durante a broncoscopia em trajetórias discretas que pudessem ser executadas pelo modelo: a respiração foi descrita como um movimento lento, cíclico e dominante, associado à expansão e relaxamento do conjunto; o batimento cardíaco, por sua vez, foi tratado como uma perturbação de menor amplitude e maior frequência, sobreposta ao movimento principal; e a tosse foi interpretada como um evento curto e abrupto, composto por uma alteração rápida da trajetória respiratória e por uma vibração localizada de curta duração.

4.4.1 Geração de trajetórias na HMI (Python/Raspberry Pi)

No Raspberry Pi, a parte escrita em Python tem uma função diferente: transformar parâmetros configuráveis em curvas, pontos e gráficos para a HMI. Este cálculo foi desenvolvido inicialmente fora da interface, para validar as curvas, os pontos e as coordenadas antes de os integrar na aplicação. Na versão para Raspberry Pi, o cálculo é feito pelo módulo `calculo_movimentos.py`. A interface permite alterar parâmetros geométricos, número de pontos e parâmetros temporais; depois, o módulo calcula os pontos de passagem e devolve gráficos 2D e 3D para visualização. Estes pontos são guardados como coordenadas X , Y e Z , podendo servir de base para atualizar posteriormente as estruturas de movimento usadas pelo firmware.

Esta lógica de cálculo evoluiu por etapas ao longo de maio de 2026: a versão inicial, a meio do mês, gerava apenas uma curva padrão genérica com suavização interna por *easing*; poucos dias depois, o código evoluiu para as três curvas específicas atualmente usadas — elipse para o batimento, parábola para a respiração e circunferência para a tosse — já preparadas para alimentar a etapa de fusão dos movimentos e a cinemática inversa dos dois manipuladores.

De forma geral, cada movimento é definido por uma curva contínua $\mathbf{p}(u)$ e por um conjunto discreto de N pontos:

$$\mathbf{p}_i = \begin{bmatrix} X_i \\ Y_i \\ Z_i \end{bmatrix} = \mathbf{p}(u_i), \quad i = 0, 1, \dots, N - 1. \quad (4.1)$$

O batimento cardíaco foi representado por uma elipse rotacionada no plano XY , uma escolha que permite produzir uma perturbação fechada, suave e de pequena amplitude, coerente com a ideia de oscilação periódica provocada pelo batimento, e que é

definida por:

$$X(u) = x_c + a \cos(u) \cos(\alpha) - b \sin(u) \sin(\alpha), \quad (4.2)$$

$$Y(u) = y_c + a \cos(u) \sin(\alpha) + b \sin(u) \cos(\alpha), \quad (4.3)$$

$$Z(u) = 0, \quad (4.4)$$

em que a e b são os semi-eixos da elipse, (x_c, y_c) é o centro da curva e α é o ângulo de rotação, podendo o sentido de percurso ser horário ou anti-horário. No conjunto de parâmetros usado nos testes da interface, foram considerados $a = 5 \text{ mm}$, $b = 2 \text{ mm}$, $x_c = y_c = 3,6 \text{ mm}$, $\alpha = 45^\circ$, $N = 10$ pontos e sentido horário.

Para a respiração foi usada uma parábola no plano ZX , por ser uma forma simples de representar uma subida progressiva com variação lenta no início e mais acentuada no final do deslocamento. Esta curva permite simular um movimento respiratório principal, controlando diretamente a altura máxima e a largura da trajetória. A mesma família de curva também pode ser usada como deslocamento rápido associado à tosse, alterando sobretudo o período e a forma como o evento é ativado. A curva base é:

$$X(u) = u, \quad (4.5)$$

$$Z(u) = \frac{H}{L^2} u^2, \quad (4.6)$$

$$Y(u) = 0, \quad 0 \leq u \leq L, \quad (4.7)$$

em que H é a altura máxima e L é a largura máxima. Quando se pretende inclinar a curva no espaço, o valor $Z(u)$ pode ser projetado por uma rotação em torno do eixo X :

$$X_{3D}(u) = X(u), \quad (4.8)$$

$$Y_{3D}(u) = Z(u) \sin(\theta), \quad (4.9)$$

$$Z_{3D}(u) = Z(u) \cos(\theta). \quad (4.10)$$

Nos valores usados nos testes, foram considerados $H = 20 \text{ mm}$, $L = 12 \text{ mm}$, $N = 10$ pontos e $\theta = 0^\circ$, tendo sido usado, para a respiração, um período $T = 4 \text{ s}$, com pausa inicial de $0,6 \text{ s}$, pausa final de $0,1 \text{ s}$ e ponto inicial igual a 5, de modo a iniciar a trajetória a partir de uma posição intermédia.

Além da alteração do movimento respiratório, a tosse inclui uma vibração curta em torno da posição de referência. Esta vibração foi representada por uma circunferência no plano XY , por permitir uma perturbação fechada e rápida, sem deslocamento vertical

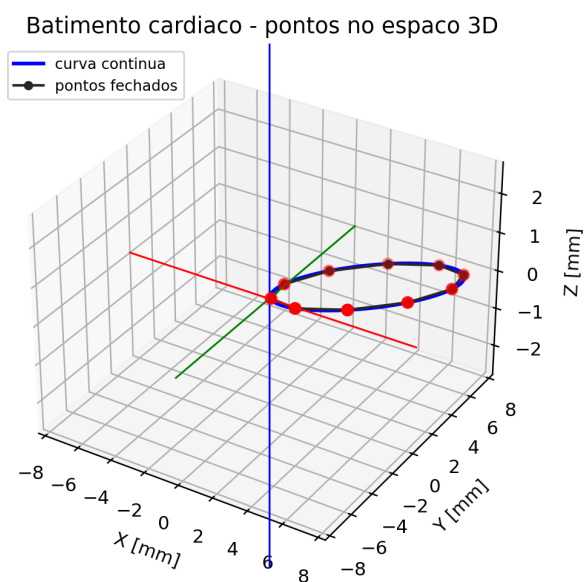
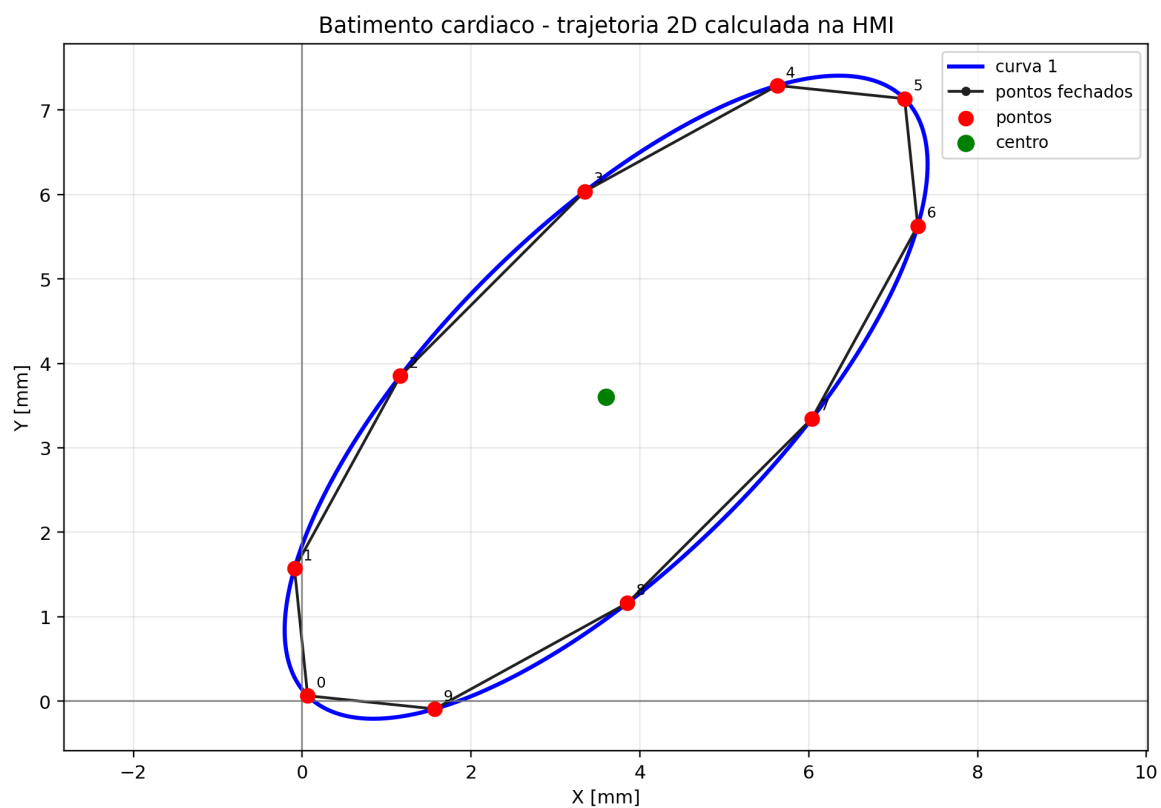


Figura 4.4: Trajetória calculada para o movimento de batimento cardíaco, com representação 2D e 3D dos pontos de passagem.

Respiracao

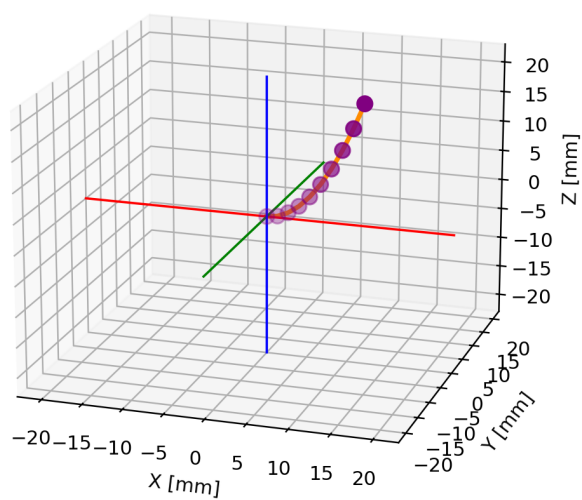
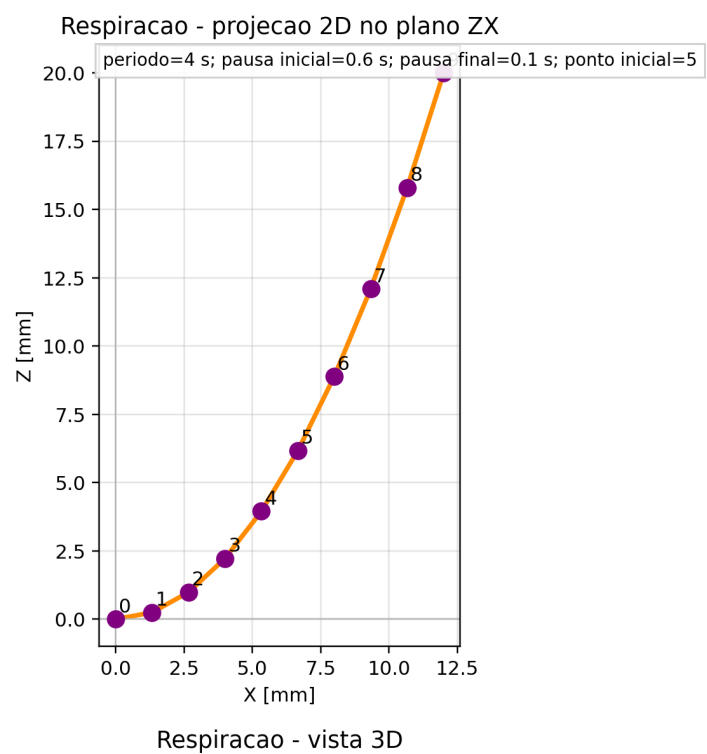


Figura 4.5: Trajetória calculada para o movimento respiratório, com representação 2D no plano ZX e visualização 3D.

adicional:

$$X(u) = r \cos(u), \quad (4.11)$$

$$Y(u) = r \sin(u), \quad (4.12)$$

$$Z(u) = 0. \quad (4.13)$$

O raio r controla a amplitude da vibração, o número de pontos controla a resolução da trajetória e o período define a rapidez do evento, tendo sido usado, no caso apresentado, $r = 2\text{ mm}$, $N = 10$ pontos, sentido horário e período de $0,5\text{ s}$.

Tabela 4.1: Parâmetros ajustáveis usados no cálculo dos movimentos na HMI.

Movimento	Parâmetros geométricos	Parâmetros temporais e de execução
Batimento cardíaco	a, b, x_c, y_c, α , número de pontos e sentido da elipse	Período, pausa inicial, pausa final e ponto inicial
Respiração	Altura H , largura L , ângulo θ e número de pontos da parábola	Período, pausa inicial, pausa final, ponto inicial e modo bidirecional
Vibração da tosse	Raio r , número de pontos e sentido da circunferência	Período, pausa inicial, pausa final e ponto inicial

4.4.2 Execução dos perfis no modelo dinâmico (C++/ESP32)

No ESP32, o firmware não tem como função principal desenhar ou recalculas as curvas geométricas dos movimentos, mas sim gerir a execução, em tempo real, de pontos de trajetória já definidos, guardados em arrays de coordenadas X , Y e Z associados a uma estrutura `MotionStorage`, na qual se guardam ainda, para cada movimento, o número de pontos, o período total, as pausas no início e no fim, o ponto inicial e a indicação de o movimento ser, ou não, bidirecional.

Esta forma de execução, comum aos três movimentos, resulta de uma evolução ao longo de vários meses: a primeira versão do firmware, em abril de 2026, executava apenas um movimento linear fixo, usado para validar a cinemática inversa; em maio, esse movimento foi substituído por perfis com base na frequência do movimento e, por fim, pela lógica genérica de `MotionStorage/MotionInstance` aqui descrita, capaz de reutilizar o mesmo mecanismo de execução para qualquer trajetória.

A separação entre `MotionStorage` e `MotionInstance` permite distinguir a definição do movimento do seu estado de execução. A primeira estrutura contém os dados fixos da trajetória; a segunda guarda o estado temporário usado durante o funcionamento, incluindo índices atuais, direção de movimento, temporizadores e estado da máquina de estados. Assim, a mesma lógica de execução pode ser aplicada à respiração, ao batimento

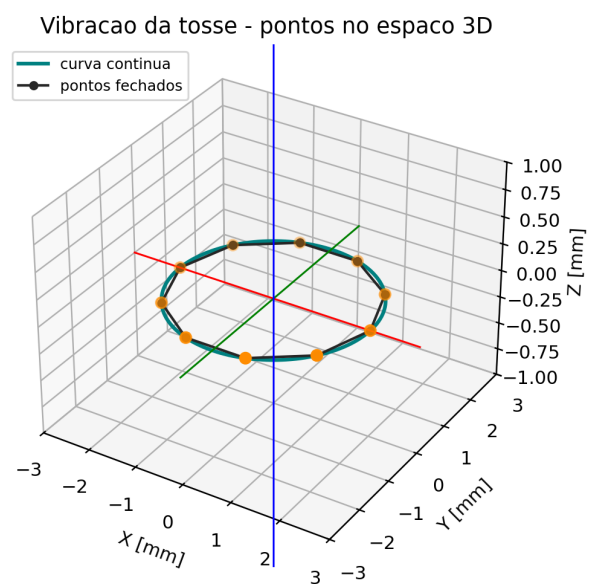
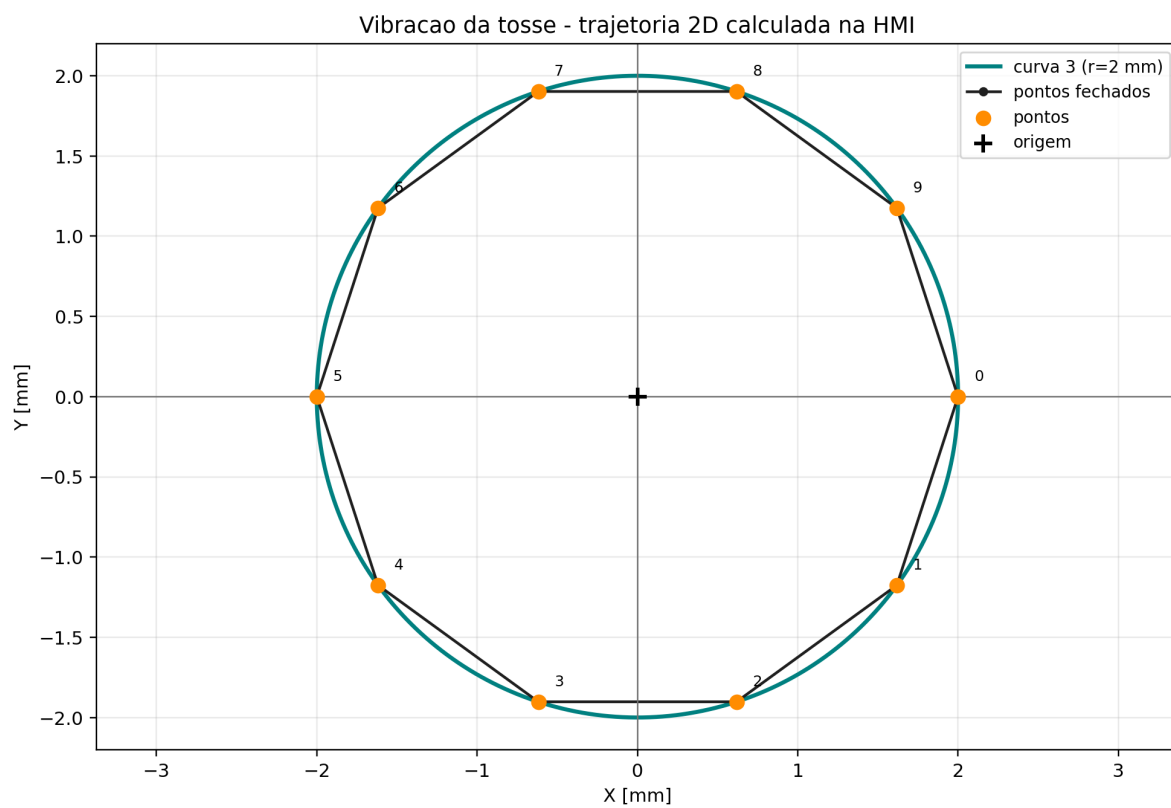


Figura 4.6: Trajetória calculada para a vibração associada à tosse, com representação 2D e 3D dos pontos de passagem.

cardíaco e à vibração da tosse.

Durante a execução, a máquina de estados percorre a sequência de pontos e atualiza os índices i e j , que representam, respetivamente, o ponto de partida e o ponto de chegada do segmento atual. Entre estes dois pontos, a posição instantânea é obtida por interpolação linear:

$$\mathbf{p}(t) = (1 - \lambda) \mathbf{p}_i + \lambda \mathbf{p}_j, \quad \lambda = \frac{t - t_i}{\Delta t}, \quad 0 \leq \lambda \leq 1. \quad (4.14)$$

O intervalo temporal entre pontos é calculado a partir do período total do movimento. Para movimentos bidirecionais, como a respiração, considera-se a ida e o regresso sobre a mesma trajetória:

$$N_{\text{seg}} = \begin{cases} 2(N - 1), & \text{movimento bidirecional,} \\ N - 1, & \text{movimento não bidirecional,} \end{cases} \quad \Delta t = \frac{T}{N_{\text{seg}}}. \quad (4.15)$$

Depois de calculada a posição intermédia, essa coordenada segue para a etapa de fusão dos movimentos e para a cinemática inversa dos manipuladores delta, pelo que a lógica em C++ trata sobretudo da execução temporal, sincronização, pausas, inversão de direção e combinação dos movimentos no modelo dinâmico.

4.5 Máquina de estados de execução dos movimentos

A função genérica de atualização de movimento percorre qualquer trajetória associada a uma `MotionInstance`. No desenvolvimento, esta rotina foi referida como `FSM_Motion_Update`. A figura 4.7 representa a sequência lógica desta máquina, mantendo os estados principais, as pausas inicial/final, a inversão de direção e o tratamento do movimento bidirecional.

A `FSM_Motion_Update()` recebe a cada ciclo de execução o sinal global de arranque `start`, a referência da instância de movimento `inst` e o instante de tempo atual `nowMs`. A função não calcula posições — essa responsabilidade pertence à etapa seguinte — mas mantém atualizados os índices de trajetória e o cronómetro interno que a função de posição instantânea vai consumir.

A conversão da lógica de execução para esta máquina de estados teve início a 18 de maio de 2026. Os primeiros testes revelaram erros de sincronização entre os índices `iIndex` e `jIndex`, corrigidos numa segunda ronda no início de junho; uma versão posterior, designada internamente por FSM 2.0 e concluída a 2 de junho de 2026, acrescentou o suporte a movimento nos dois sentidos e a pausas configuráveis no início e no fim de cada trajetória.

Interação com as estruturas de dados. A cada chamada, a FSM lê da `MotionStorage` associada (via `inst.def`): o número de pontos `n.Points`, o período total `period`, o intervalo por segmento `dtMs`, o índice inicial `start_Index`, o flag `bidirectional` e as durações de pausa `Dt_pauseMs_Ini/Dt_pauseMs_End`. Escreve na `MotionInstance`: os índices `iIndex` e `jIndex`, os cronómetros `elapsed_1`, `elapsed_2` e `segmentStartMs`, a direção `Direction` e a flag `Executing`.

Tabela 4.2: Pontos calculados para os movimentos usados nos testes da interface.

Ponto	Batimento cardíaco			Respiração			Vibração da tosse		
	X	Y	Z	X	Y	Z	X	Y	Z
0	0,06	0,06	0,00	0,00	0,00	0,00	2,00	0,00	0,00
1	-0,09	1,57	0,00	1,33	0,00	0,25	1,62	-1,18	0,00
2	1,16	3,85	0,00	2,67	0,00	0,99	0,62	-1,90	0,00
3	3,35	6,04	0,00	4,00	0,00	2,22	-0,62	-1,90	0,00
4	5,63	7,29	0,00	5,33	0,00	3,95	-1,62	-1,18	0,00
5	7,14	7,14	0,00	6,67	0,00	6,17	-2,00	0,00	0,00
6	7,29	5,63	0,00	8,00	0,00	8,89	-1,62	1,18	0,00
7	6,04	3,35	0,00	9,33	0,00	12,10	-0,62	1,90	0,00
8	3,85	1,16	0,00	10,67	0,00	15,80	0,62	1,90	0,00
9	1,57	-0,09	0,00	12,00	0,00	20,00	1,62	1,18	0,00

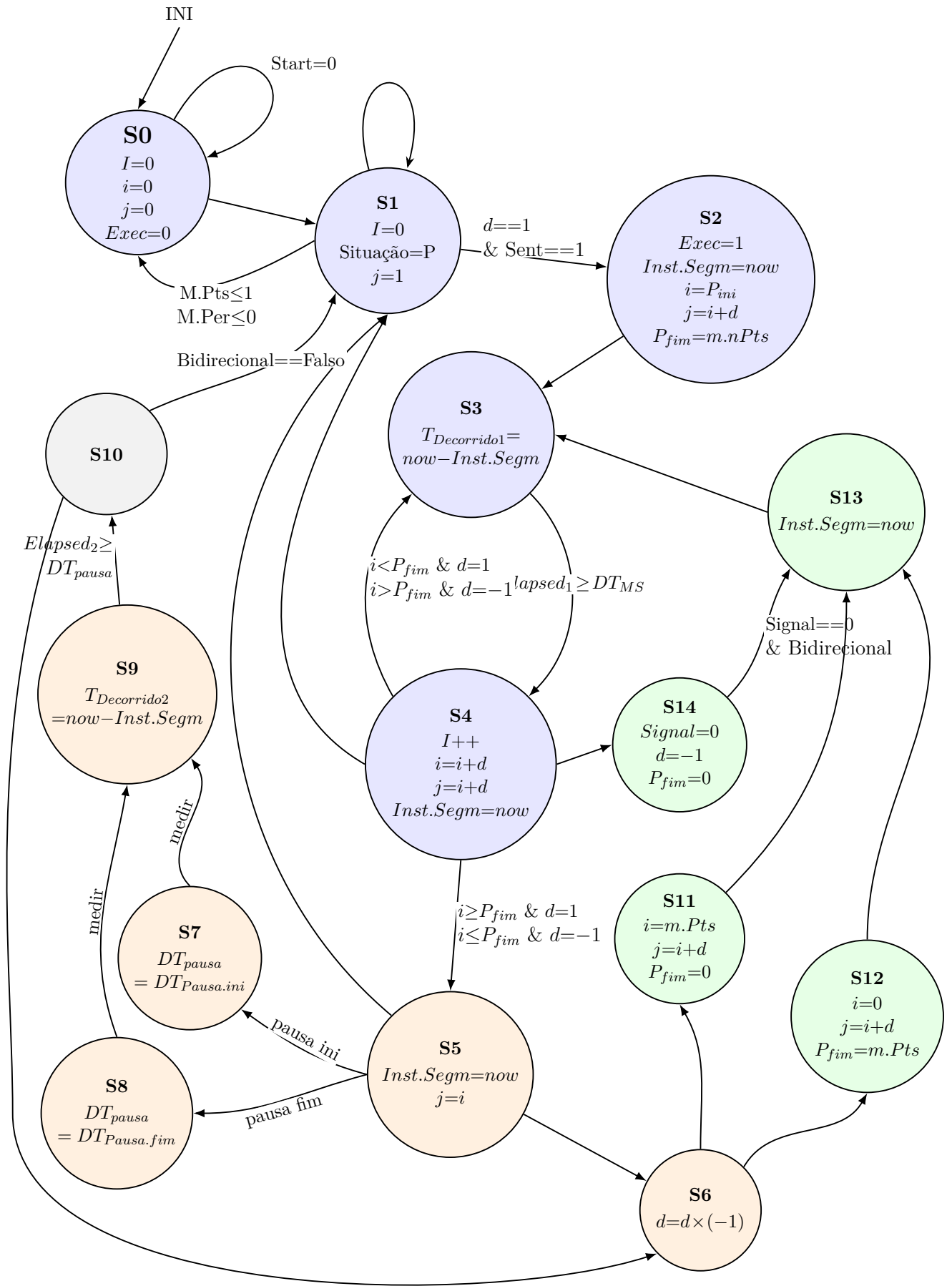


Figura 4.7: Esquema da FSM_Motion_Update.

Fases de funcionamento.

- **Arranque (S0→S1→S2):** No estado S0 a instância está inativa, com índices e `Executing` zerados. Quando `start=1` e a instância está marcada como ativa, a FSM transita para S1, que valida os parâmetros da trajetória (pelo menos dois pontos e período positivo) e inicializa `Direction=1`. Em S2 regista `segmentStartMs=nowMs`, define `iIndex=start_Index` e `jIndex=iIndex+d`, e transita imediatamente para S3.
- **Ciclo de segmentos (S3↔S4):** S3 mede o tempo decorrido (`elapsed_1 = nowMs - segmentStartMs`) e aguarda que este iguale `dtMs`. Quando atingido, S4 avança `iIndex` na direção `d` (+1 ou -1), atualiza `jIndex=iIndex+d`, reinicia o cronómetro e retorna a S3. Este ciclo repete-se até `iIndex` atingir `End_Point`.
- **Gestão do extremo e pausas (S5→S7/S8→S9→S10):** Ao atingir o fim da passagem, S5 decide o caminho: se existir pausa configurada, S7 ou S8 carregam a duração correspondente (inicial ou final), S9 aguarda até ao fim da pausa, e S10 decide se recomeça o ciclo bidirecional (→S6) ou reinicializa do início (→S1).
- **Inversão de direção (S6→S11/S12→S13):** S6 inverte `Direction` (multiplica por -1). Consoante o novo sentido, S11 (passagem descendente) ou S12 (passagem ascendente) reinicializa os índices para o extremo correto. S13 regista o novo `segmentStartMs` e entra de imediato em S3.
- **Interrupção de tosse (S4→S14→S13):** Se o sinal global `Tosse_signal` for ativado durante o avanço (S4), a FSM entra em S14, que força a direção de regresso (`Direction=-1`, `End_Point=0`) e encadeia com S13 para iniciar a descida de emergência.

Saída para a função de posição instantânea. Ao terminar cada chamada, a `MotionInstance` contém: `iIndex` (índice do ponto de partida do segmento atual), `jIndex` (índice do ponto de chegada) e `elapsed_1` (tempo decorrido desde o início do segmento). A função `Intermed_position()` lê estes três campos, acede ao `MotionStorage` para obter as coordenadas `X[i]`, `X[j]`, `Y[i]`, `Y[j]`, `Z[i]`, `Z[j]` e o intervalo `dtMs`, e calcula a posição interpolada entre os dois pontos. O resultado é um triploto (X, Y, Z) em coordenadas relativas ao ponto de referência `Index0`, que segue depois para a função de fusão e cinemática inversa.

A posição instantânea é obtida por interpolação linear entre dois pontos consecutivos da trajetória:

$$P(\alpha) = P_i + (P_j - P_i) \alpha$$

Definindo a razão temporal bruta $\alpha_0 = t_{dec}/DT_{MS}$, onde t_{dec} é o valor de `elapsed_1` e

DT_{MS} é $\mathbf{dtMs}$, o coeficiente de interpolação saturado é:

$$\alpha = \begin{cases} 0 & \text{se } \alpha_0 < 0 \\ \alpha_0 & \text{se } 0 \leq \alpha_0 \leq 1 \\ 1 & \text{se } \alpha_0 > 1 \end{cases} \quad \alpha_0 = \frac{t_{dec}}{DT_{MS}}$$

4.6 Cinemática inversa do manipulador delta

A cinemática inversa converte a posição desejada da plataforma móvel (X, Y, Z) nos três ângulos de servo θ_1 , θ_2 e θ_3 . A figura 4.8 mostra a representação lateral usada como referência para as variáveis do cálculo: L_1 representa o braço acionado pelo servo, L_2 o braço passivo, L_3 o afastamento horizontal até à articulação da plataforma móvel, ϕ e ω são ângulos intermédios e θ é o ângulo final aplicado ao servo.

Nas primeiras experiências com este problema, antes de se fixar a abordagem final, foram consultados dois recursos que representam uma família de solução diferente: a biblioteca *Arduino Delta-Kinematics-Library* [14] e o tutorial de Nikanorov [15]. Ambos resolvem a cinemática inversa por interseção geométrica entre uma circunferência, centrada no eixo do servo e com raio igual ao braço ativo, e uma esfera centrada na plataforma com raio igual ao braço passivo, parametrizando o problema com um raio de base (f) e um raio de plataforma (e) em vez dos comprimentos L_1 , L_2 e L_3 usados neste projeto, e obtendo o ângulo final diretamente por atan2 da interseção, sem uma etapa explícita de lei dos cossenos. Esta abordagem foi testada nos primeiros esboços de código, mas revelou-se menos direta de adaptar à disposição mecânica dos servos e ao referencial lateral usados no desenho dos manipuladores deste projeto, pelo que se optou por explorar, em alternativa, a decomposição por lei dos cossenos seguida de atan2 — matematicamente equivalente, mas mais fácil de mapear diretamente às variáveis `SERVO_OFFSET_X/SERVO_OFFSET_Z` do desenho mecânico.

A primeira versão funcional deste cálculo foi implementada no final de abril de 2026, com base num repositório de código aberto de um manipulador delta impresso em 3D e controlado por Arduino [16], aplicada isoladamente a um único braço do manipulador e validada através de um movimento linear simples. A meio de maio, a função foi generalizada para servir os dois manipuladores delta a partir da mesma base de código, eliminando a duplicação que existia entre os dois braços e permitindo a sincronização dos respetivos movimentos. Esta abordagem geométrica — decomposição por braço com simetria de 120° , resolvendo cada perna pela lei dos cossenos seguida de atan2 — segue a família de soluções estabelecida por Zsombor-Murray para a cinemática do robô delta de Clavel [17], e mantém-se consistente com formulações mais recentes do mesmo problema [18].

O firmware aplica o mesmo cálculo aos três braços. A diferença entre eles está

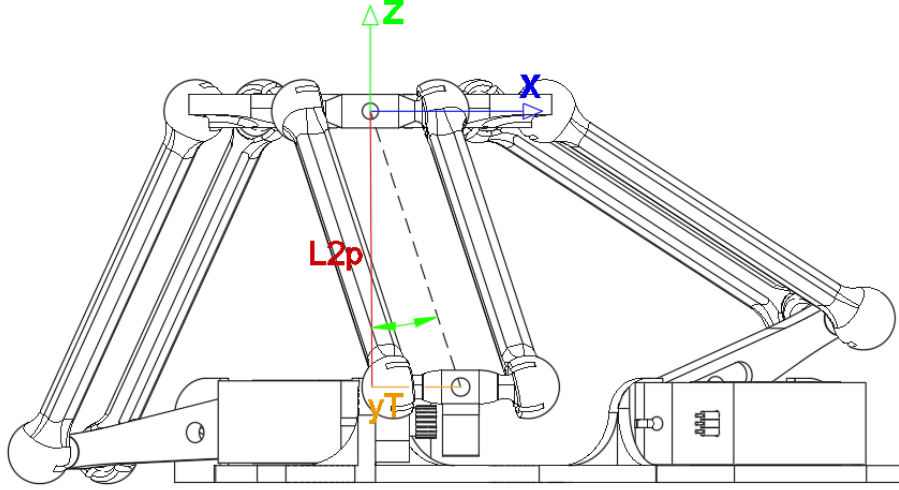


Figura 4.8: Representação lateral das variáveis usadas na cinemática inversa do manipulador delta.

apenas na rotação do referencial no plano XY . Para cada manipulador existe um offset global α_m , definido na estrutura `Manipulador_Config`, e para cada servo existe uma rotação adicional β_k :

$$\alpha_m = \begin{cases} -15^\circ, & \text{Manipulador Delta 1} \\ -45^\circ, & \text{Manipulador Delta 2} \end{cases} \quad \beta_k \in \{0^\circ, 120^\circ, 240^\circ\}$$

$$\varphi_k = \alpha_m + \beta_k \quad R_z(\varphi_k) = \begin{bmatrix} \cos \varphi_k & -\sin \varphi_k & 0 \\ \sin \varphi_k & \cos \varphi_k & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Assim, o ponto pedido é primeiro expresso no plano de trabalho do servo k :

$$\begin{bmatrix} x'_k \\ y'_k \\ z'_k \end{bmatrix} = R_z(\varphi_k) \begin{bmatrix} X \\ Y \\ Z \end{bmatrix} - \begin{bmatrix} 0 \\ 0 \\ z_{offset} \end{bmatrix} \implies \begin{cases} x'_k = X \cos \varphi_k - Y \sin \varphi_k \\ y'_k = X \sin \varphi_k + Y \cos \varphi_k \\ z'_k = Z - z_{offset} \end{cases}$$

No código, $z_{offset} = 0$ mm, mas a variável foi mantida para permitir calibração posterior.

Depois da rotação, o problema fica reduzido ao plano lateral do servo. A extremidade associada à plataforma móvel é deslocada pelo comprimento L_3 :

$$x_{arm,k} = x'_k + L_3$$

Como o braço passivo L_2 pode sair ligeiramente do plano, o código calcula a sua projeção nesse plano. Esta projeção só é válida se a junta esférica não ultrapassar o limite angular

admitido:

$$L_{2p,k} = \sqrt{L_2^2 - y_k'^2} \quad \gamma_k = \arcsin\left(\frac{y_k'}{L_2}\right) \quad |\gamma_k| < 0,5934 \text{ rad} \approx 34^\circ$$

Este limite aparece no firmware como proteção para evitar que o ângulo entre a junta esférica e o braço passivo saia da zona mecânica admissível.

De seguida calcula-se a distância entre o eixo do servo e a articulação projetada. Esta distância é designada no código por `ext`:

$$d_k = \sqrt{z_k'^2 + (d_s - x_{arm,k})^2} \quad d_s = 32 \text{ mm}$$

Antes de calcular o ângulo, o firmware confirma se os comprimentos L_1 , $L_{2p,k}$ e d_k conseguem formar um triângulo:

$$L_{2p,k} - L_1 < d_k < L_1 + L_{2p,k}$$

Se esta condição falhar, a posição pretendida fica fora do alcance geométrico daquele braço e a função de cinemática inversa devolve erro.

Com a geometria validada, o ângulo interno ϕ_k é obtido pela lei dos cossenos, enquanto ω_k corresponde à inclinação da reta entre o pivô do servo e a articulação projetada:

$$\phi_k = \arccos\left(\frac{L_1^2 + d_k^2 - L_{2p,k}^2}{2L_1d_k}\right) \quad \omega_k = \text{atan2}(z_k', d_s - x_{arm,k})$$

O ângulo enviado ao servo é então:

$$\theta_k = \phi_k + \omega_k \quad 45^\circ \leq \theta_k \leq 225^\circ$$

O intervalo de 45° a 225° é o limite angular definido no firmware por `SERVO_ANGLE_MIN` e `SERVO_ANGLE_MAX`, pelo que, se algum dos três ângulos calculados ficar fora deste intervalo, a posição é rejeitada e os servos não são atualizados.

Por fim, cada θ_k é convertido para largura de pulso PWM por interpolação linear:

$$PWM_k = PWM_{k,min} + \frac{\theta_k - \theta_{min}}{\theta_{max} - \theta_{min}} (PWM_{k,max} - PWM_{k,min})$$

Considerando os valores de referência $PWM_{min} = 500 \mu s$ e $PWM_{max} = 2500 \mu s$, a relação é crescente no Manipulador Delta 1 e invertida no Manipulador Delta 2, devido à montagem mecânica oposta dos servos. Esta inversão é feita no arranque do firmware, quando os limites `thetaMinPulse` e `thetaMaxPulse` do segundo manipulador são trocados.

Os limites de PWM usados no código não foram assumidos apenas a partir de valores teóricos do servo. Estes valores foram calibrados experimentalmente, começando

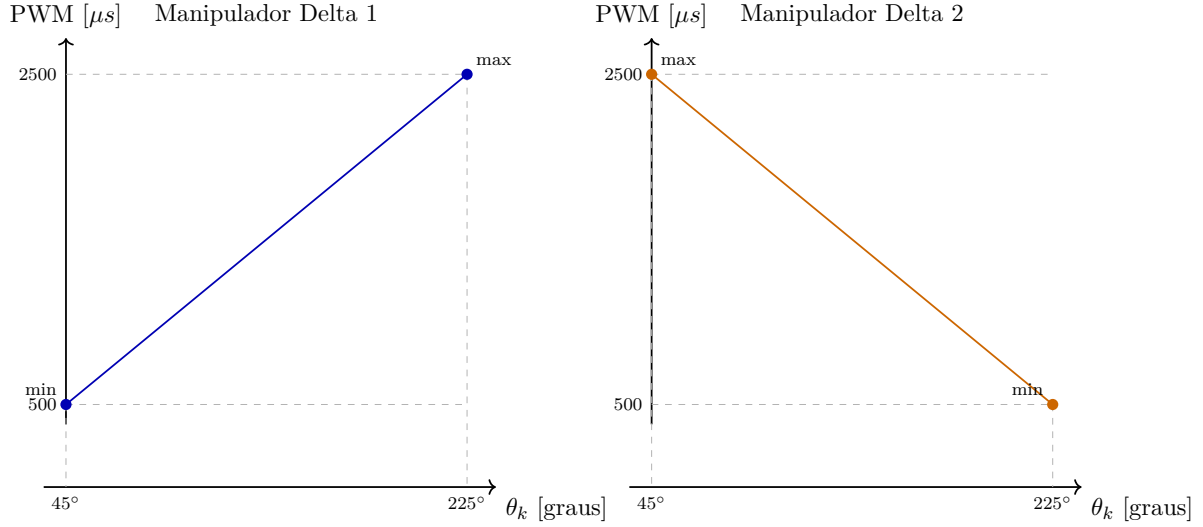


Figura 4.9: Relação linear de referência entre o ângulo calculado pela cinemática inversa e o pulso PWM aplicado aos servos nos dois manipuladores.

pelo alinhamento da patilha de cada servo com a base onde estava acoplado. O objetivo deste alinhamento era garantir que as posições físicas de 0° e 180° ficavam paralelas à superfície da base. Depois desse alinhamento mecânico, era feita a transição manual da referência física para o intervalo usado pela cinemática: no Manipulador Delta 1, de 0° para 45° ; no Manipulador Delta 2, devido à orientação oposta dos servos, de 180° para 45° . Por fim, foi necessário repetir ensaios de calibração para confirmar que os impulsos PWM correspondiam aos limites de 45° e 225° , admitindo uma tolerância aproximada de $\pm 5^\circ$.

$$\text{Manipulador Delta 1 - Calibrado} - \left\{ \begin{array}{ll} \text{Mínimo} & \text{Máximo} \\ \text{Servo } \theta_1 : & 520 \mu s \quad 2520 \mu s \\ \text{Servo } \theta_2 : & 550 \mu s \quad 2550 \mu s \\ \text{Servo } \theta_3 : & 560 \mu s \quad 2560 \mu s \end{array} \right.$$

$$\text{Manipulador Delta 2 - Calibrado} - \left\{ \begin{array}{ll} \text{Mínimo} & \text{Máximo} \\ \text{Servo } \theta_1 : & 460 \mu s \quad 2460 \mu s \\ \text{Servo } \theta_2 : & 470 \mu s \quad 2470 \mu s \\ \text{Servo } \theta_3 : & 420 \mu s \quad 2420 \mu s \end{array} \right.$$

No arranque, o firmware mantém estes intervalos físicos para associar cada servo ao seu canal PWM, mas inverte a correspondência entre `thetaMinPulse` e `thetaMaxPulse` no Manipulador Delta 2. Assim, a cinemática inversa continua a gerar os mesmos três ângulos lógicos, enquanto a conversão final para PWM respeita a montagem, o sentido de

rotação e a calibração individual de cada servo.

4.7 Comunicação série e comandos

A comunicação série é organizada por duas máquinas de estados principais: a primeira é a `FSM_Serial_reader()`, responsável apenas pela recepção dos dados que chegam pela porta série, e a segunda é a `FSM_Serial_Command()`, que interpreta esses dados e altera o comportamento do sistema, sendo esta separação usada precisamente para evitar que a leitura da porta série ficasse misturada com a lógica de controlo dos movimentos.

A primeira versão desta separação em duas máquinas de estados ficou operacional a 8 de junho de 2026. Nas semanas seguintes, à medida que a interface gráfica do Raspberry Pi ia sendo desenvolvida, os comandos disponíveis foram sendo atualizados e ajustados para corresponder às ações que a HMI passou a expor ao utilizador.

A `FSM_Serial_reader()` funciona como uma camada de aquisição. Quando o sistema está em modo simples, lê um único carácter e coloca esse valor em `CH_Command`. Quando é ativado o modo de comando longo, através do comando `W`, passa a preencher o vetor `ST_Bus` até receber o fim da mensagem. Só depois de a mensagem estar completa é que ativa a flag de linha pronta. Assim, a máquina de comandos nunca precisa de ler diretamente da porta série; apenas consome comandos já preparados.

A `FSM_Serial_Command()` funciona como a camada de decisão. Esta máquina verifica se existe um comando disponível, interpreta o carácter ou a cadeia recebida, atualiza flags como `start_comand`, `Safe_comand` e `sinai_tossir`, seleciona que movimentos ficam ativos e, nos comandos longos, altera parâmetros das estruturas `MotionStorage`. Usar duas máquinas de estados torna o comportamento mais previsível: a recepção pode continuar a ser tratada passo a passo, enquanto a interpretação dos comandos fica concentrada numa rotina própria.

Os principais comandos simples disponíveis são:

Tabela 4.3: Comandos simples disponíveis na comunicação série.

Comando	Função
S	Inicia a execução do ciclo de movimentos.
P	Para a execução e calcula a duração do ensaio.
L	Entra no menu de leitura, teste e depuração.
M	Entra no menu de seleção do modo fisiológico.
W	Ativa o modo de comandos com vários caracteres.

No menu L, o segundo carácter define a ação a executar:

No menu M, o segundo carácter escolhe que movimentos ficam ativos:

Os comandos longos começam por `W` e permitem enviar uma cadeia de caracteres para alterar parâmetros numéricos. No estado atual do firmware, a estrutura implemen-

Tabela 4.4: Subcomandos do menu de leitura, teste e depuração.

Comando	Função
L1	Ativa o comando de segurança definido por <code>Safe_comand = 1</code> .
L2	Desativa esse comando de segurança, colocando <code>Safe_comand = 0</code> .
L3	Ativa o sinal de tosse, através de <code>sinal_tossir = 1</code> .
Lr	Imprime os dados de respiração, <code>Resp_move</code> e <code>Resp_inst</code> .
Lb	Imprime os dados de batimento, <code>Bat_move</code> e <code>Bat_inst</code> .
Lt	Reservado para leitura dos dados de tosse.
La	Imprime em conjunto os dados principais de respiração e batimento.

Tabela 4.5: Subcomandos de seleção do modo fisiológico.

Comando	Modo selecionado
Ma	Ativa respiração, batimento e tosse.
Mh	Ativa o modo humano, com respiração e batimento.
Mr	Ativa apenas a respiração.
Mb	Ativa apenas o batimento.
Ms	Desativa os movimentos, deixando o sistema em modo estático.

tada permite selecionar o movimento de destino com R, B ou T, correspondentes a respiração, batimento e tosse, e depois alterar o período com o campo `T[valor]`. Por exemplo, uma mensagem do tipo `WdRT[4.0]` é interpretada como uma alteração do período da respiração para 4,0 s, de acordo com o formato que a máquina de estados procura no vetor `ST_Command`. Os campos para pausa e alteração de pontos de movimento aparecem previstos na máquina, mas ainda não estão totalmente desenvolvidos no código.

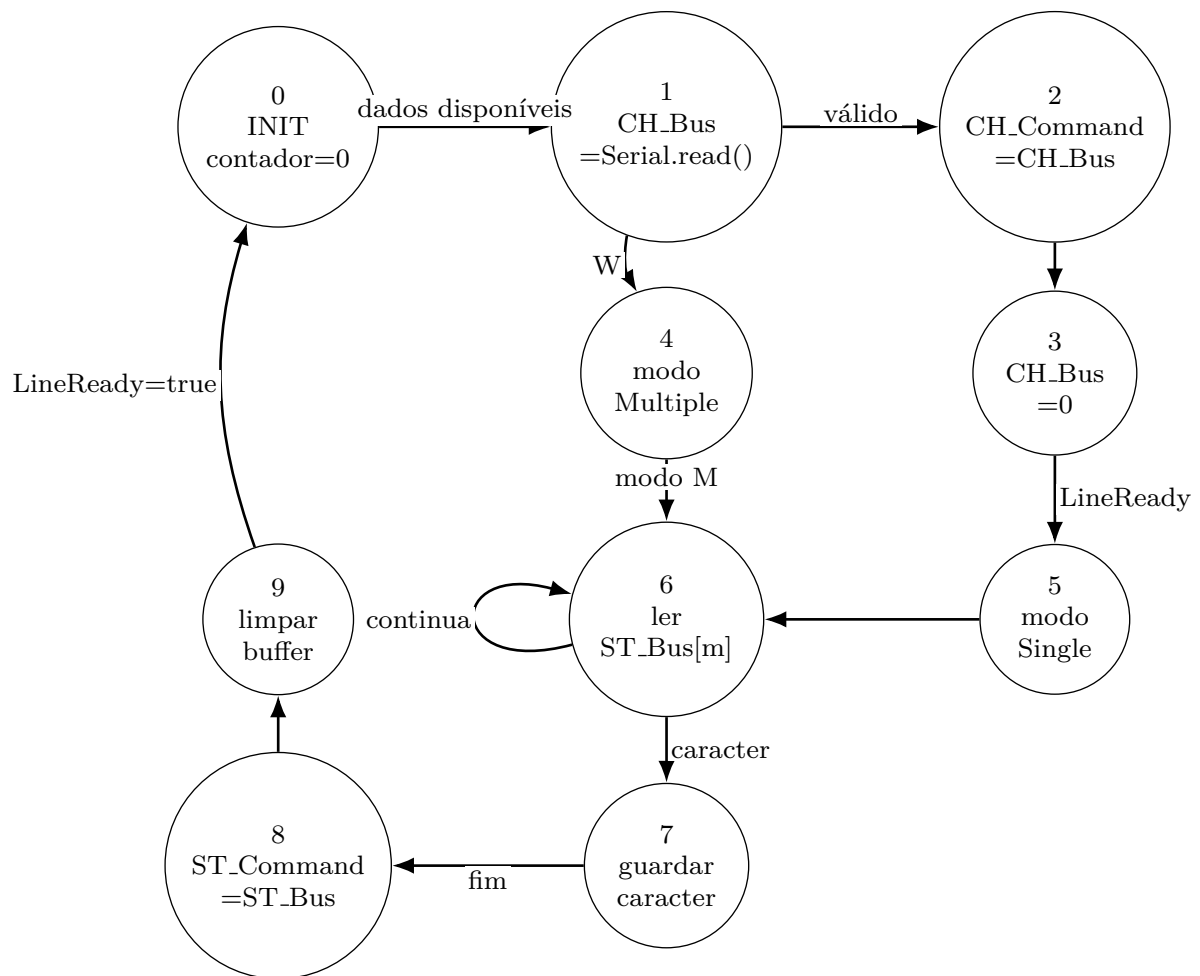


Figura 4.10: FSM.Serial_Read responsável pela leitura e validação dos dados recebidos por comunicação série.

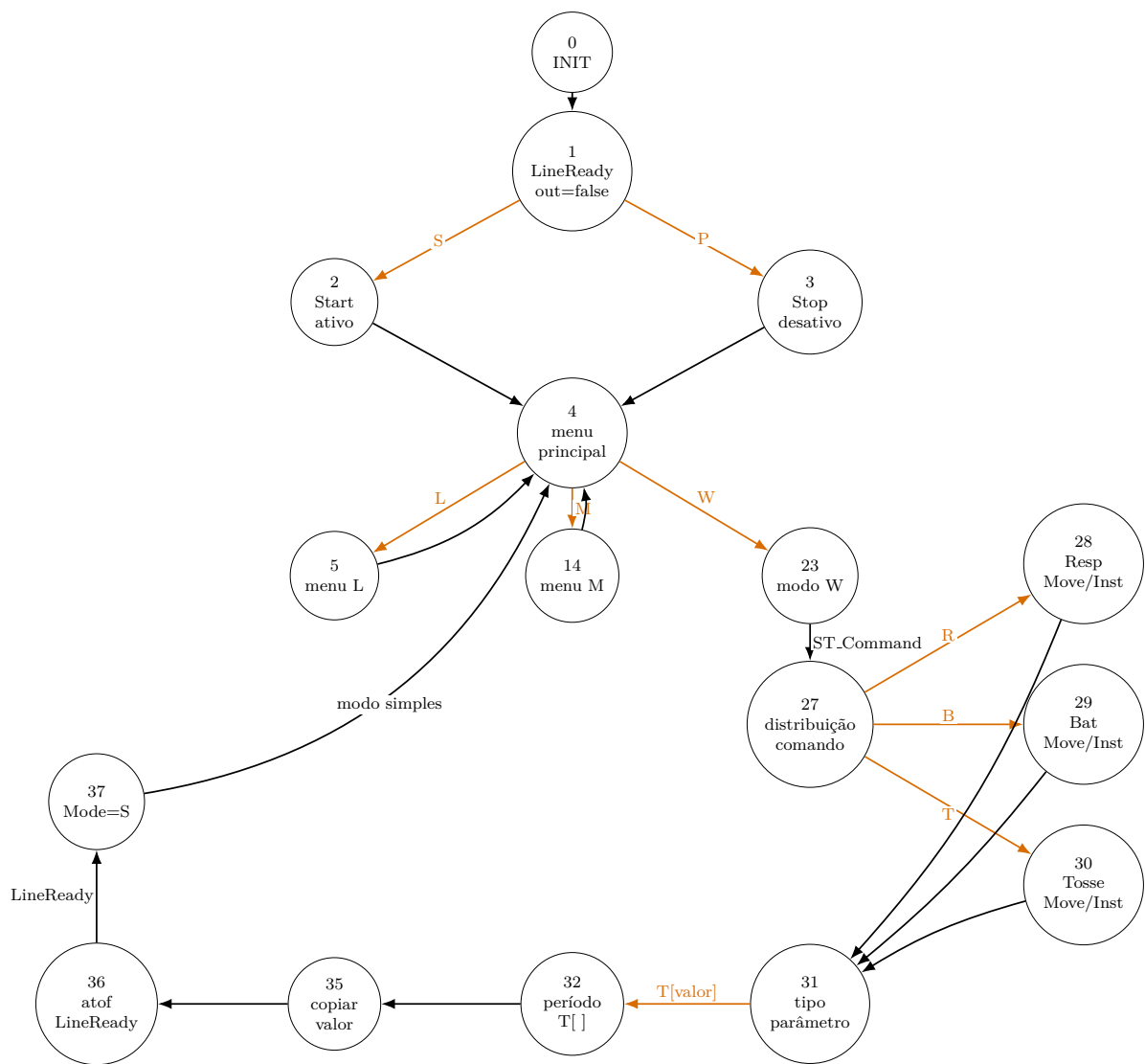


Figura 4.11: FSM_Serial_Command responsável pela interpretação dos comandos recebidos por comunicação série.

4.8 Interface HMI no Raspberry Pi

A interface homem-máquina foi desenvolvida para transformar os comandos de baixo nível do firmware numa utilização mais simples e natural, já que, sem esta camada, o utilizador teria de memorizar comandos como **S**, **P**, **Mr**, **Mb**, **Mh** ou **Ma**, escrevendo-os diretamente no monitor série; com a HMI, essas ações passam antes a estar disponíveis através de botões, menus e páginas de configuração, reduzindo a probabilidade de erro e tornando o modelo mais adequado para demonstração e treino.

A primeira versão funcional desta interface, identificada internamente como `Interface_V1`, ficou disponível a 19 de junho de 2026. Nas semanas seguintes foi sujeita a várias rondas de revisão: primeiro ao nível da comunicação série e da organização do repositório (19–26 de junho), depois com modificações à disposição gráfica em modo cliente e DEV (26 de junho) e, já na reta final do projeto, com a correção de erros na troca de dados entre a interface e o firmware e melhorias adicionais à página de monitorização (1–3 de julho de 2026).

Na versão para Raspberry Pi, o ficheiro principal da interface é `RB_PI4B_Main_PI.py`. A aplicação usa Flask para servir as páginas web e Flask-SocketIO para enviar atualizações em tempo real aos browsers ligados. Esta arquitetura permite ter uma interface local no monitor do Raspberry Pi e, ao mesmo tempo, uma interface móvel acessível por telemóvel ou tablet na mesma rede. Para facilitar esse acesso, a aplicação gera um endereço local e um código QR que aponta para a rota `/mobile`.

A comunicação com o ESP32 é feita por uma porta série a 115200 baud. A interface tenta detetar automaticamente a porta mais provável no Linux, dando prioridade a dispositivos em `/dev/serial/by-id`, `/dev/ttyUSB*` e `/dev/ttyACM*`. A leitura e a escrita série são executadas em threads separadas: uma thread lê continuamente as mensagens recebidas do ESP32 e atualiza o histórico, enquanto outra consome uma fila de transmissão e envia os comandos pedidos pela interface. Esta separação evita que a interface fique bloqueada durante leituras ou esperas da porta série.

O estado da aplicação é guardado numa estrutura partilhada que contém, entre outros campos, o modo de desenvolvimento, o histórico da comunicação série, o estado de arranque/paragem e o modo fisiológico ativo. Sempre que algum destes valores muda, a função de atualização emite um evento SocketIO para todos os browsers ligados. Assim, se o utilizador abrir simultaneamente a interface no Raspberry Pi e no telemóvel, ambos os ecrãs podem refletir o mesmo estado de funcionamento.

As ações principais da HMI são:

- **Start/Stop:** os botões de arranque e paragem enviam, respetivamente, os comandos **s** e **p** para o ESP32, alterando também o estado visual da interface;
- **Seleção de modo:** o menu de modos converte escolhas como estático, respiração, coração, humano e completo nos comandos série **Ms**, **Mr**, **Mb**, **Mh** e **Ma**;

- **Ligação série:** a interface permite ligar, desligar e tentar reconectar a comunicação com o ESP32, mostrando o estado como desligado, a ligar, ligado ou erro;
- **Monitor série:** em modo de desenvolvimento, o utilizador pode observar as mensagens recebidas do ESP32 e enviar comandos manuais, o que mantém uma via de depuração direta;
- **Modo cliente e modo DEV:** o modo cliente apresenta apenas as ações necessárias à utilização normal, enquanto o modo DEV, protegido por palavra-passe, revela ferramentas de depuração, envio manual de comandos e encerramento completo da aplicação;
- **Configurações de movimentos:** a interface permite editar parâmetros associados à respiração, batimento e tosse, guardar configurações em ficheiro JSON e gerar pontos de movimento a partir do módulo `calcula_movimentos.py`.

A página de configurações de movimentos usa parâmetros geométricos para gerar curvas de referência. A curva elíptica é usada para representar o batimento cardíaco, a curva parabólica no plano ZX é usada para movimentos de respiração/tosse e a circunferência no plano XY é usada para vibração da tosse. O módulo de cálculo devolve gráficos 2D e 3D em formato de imagem codificada em base64, permitindo que os resultados sejam apresentados diretamente na interface web sem abrir janelas externas. Os pontos calculados são guardados nas configurações e podem servir de base para a atualização posterior dos movimentos no firmware.

O desenvolvimento da HMI foi feito depois de validadas duas camadas de suporte. A primeira foi a comunicação série elementar, testada com scripts Python que liam mensagens do ESP32, enviavam comandos escritos no terminal e confirmavam o efeito dos sinais DTR/RTS no reset da placa. A segunda foi o cálculo isolado das trajetórias, testando a geração de pontos e gráficos antes da sua integração na aplicação Flask. Esta ordem de desenvolvimento reduziu a incerteza: quando a interface foi construída, já estava confirmado que o canal série e a lógica de geração de movimentos funcionavam de forma independente.

4.9 Preparação e sincronização da tosse

Esta funcionalidade foi implementada numa fase avançada do desenvolvimento, a partir de 24 de junho de 2026, depois de os movimentos de respiração e batimento cardíaco já estarem validados de forma independente — o que permitiu tratar a tosse como uma alteração temporária a um comportamento já existente, em vez de partir do zero.

A função de preparação da tosse é responsável por transformar um pedido de tosse num evento fisiológico sobreposto à respiração. Quando o sinal de tosse é ativado, a

rotina guarda os parâmetros normais da respiração, reduz temporariamente o período, ajusta as pausas e aumenta a amplitude em Z . Em paralelo, ativa a instância de vibração ou perturbação da tosse.

Quando o ciclo respiratório atinge o estado apropriado para terminar o evento, a rotina repõe o período original, as pausas e o multiplicador de amplitude. Esta estratégia permite que a tosse não seja apenas uma trajetória independente, mas uma alteração temporária do comportamento respiratório, mais coerente com o fenômeno fisiológico que se pretende simular.

4.10 Depuração e validação interna

A validação interna do firmware foi feita em paralelo com o desenvolvimento das máquinas de estados. Como o sistema junta comunicação série, geração de trajetórias, cinemática inversa e acionamento de seis servos, foi necessário criar pontos de observação que permitissem perceber em que parte da cadeia surgia um erro. Para isso foram usadas mensagens enviadas pela porta série e funções auxiliares de impressão, principalmente `debugPrintMove()` e `debugPrintInstance()`.

A função `debugPrintMove()` permite confirmar os dados guardados numa estrutura `MotionStorage`: número de pontos, período, intervalo temporal entre pontos e coordenadas X , Y e Z de cada posição. Esta verificação foi importante para confirmar se os movimentos de respiração, batimento e tosse estavam a ser carregados com os valores esperados antes de serem usados pela interpolação. A função `debugPrintInstance()` foi usada para observar o estado de execução de cada movimento, incluindo se a instância estava ativa, qual o índice atual e qual a estrutura de movimento associada.

Foram também mantidos testes isolados para componentes específicos. Os testes ao LED RGB permitiam confirmar rapidamente o estado dos comandos recebidos, uma vez que diferentes comandos alteravam a cor do LED. Os testes aos servos, por sua vez, ajudavam a verificar se os canais PWM estavam corretamente associados, se os limites calibrados eram respeitados e se os servos respondiam no sentido esperado. Esta validação por partes foi essencial para distinguir erros de software, como uma transição incorreta numa máquina de estados, de problemas de montagem ou calibração.

Outro aspeto observado durante os ensaios foi o comportamento da porta série no ESP32-S3. Ao abrir o monitor série ou alguns scripts Python, os sinais DTR/RTS podem provocar o reset da placa. Quando isso acontece, o `setup()` volta a ser executado, os servos são novamente associados aos canais PWM e o sistema regressa ao estado inicial. Por este motivo, durante os testes é necessário ter em conta que a abertura da porta série pode alterar temporariamente o estado do sistema, mesmo sem ter sido enviado um comando de movimento.

Assim, a depuração não foi usada apenas para imprimir valores no terminal, mas

como uma ferramenta de validação do fluxo completo: receção do comando, interpretação pela máquina de estados, atualização das estruturas de movimento, cálculo da posição intermédia, aplicação da cinemática inversa e envio final dos impulsos PWM para os servos.

Esta prática acompanhou praticamente todo o desenvolvimento do firmware, tendo-se tornado particularmente intensa na fase final do projeto, entre o final de junho e o início de julho de 2026, quando a integração entre o firmware e a interface gráfica do Raspberry Pi expôs a maioria dos erros de sincronização e de formatação dos comandos série.

Capítulo 5

Testes e Análise de Resultados

Nota sobre o estado atual do capítulo

Este capítulo encontra-se em desenvolvimento por falta de informação experimental completa, uma vez que, nesta fase do relatório, ainda não existem resultados quantitativos suficientes para apresentar uma análise final de desempenho. Assim, o capítulo descreve o plano de testes, os critérios de aceitação e a informação que deve ser recolhida nas próximas iterações do projeto.

5.1 Plano de testes

O plano de testes foi definido para validar separadamente o modelo mecânico, a eletrónica, o firmware e a integração completa, seguindo uma estratégia que consiste em testar primeiro cada subsistema de forma isolada e, depois, combinar progressivamente respiração, batimento cardíaco e tosse.

Para cada ensaio devem ser registados: configuração usada, período, amplitude, número de pontos da trajetória, modo de operação, comportamento observado, eventuais falhas, temperatura dos atuadores, estabilidade da alimentação e repetibilidade do movimento. A comparação deve ser feita com os valores-alvo definidos para os movimentos representados no protótipo.

5.2 Testes do modelo físico

Os primeiros testes devem confirmar se o modelo impresso mantém a geometria pretendida e se permite a passagem adequada do broncoscópio, observando-se ainda deformações, zonas frágeis, arestas, dificuldade de limpeza e resistência ao manuseamento. No caso de materiais flexíveis, deve também ser avaliada a relação entre flexibilidade e estabilidade dimensional.

O teste inicial em PLA serve como prova de conceito da geometria e da montagem, ainda que, por ser um material rígido, não represente a solução final para realismo mecânico. Por isso, ensaios posteriores com TPU, TPE ou outros filamentos flexíveis devem comparar facilidade de impressão, acabamento, deformação elástica e resistência à repetição dos movimentos.

5.3 Testes dos atuadores e da estrutura delta

Os servos devem ser testados antes da integração com a árvore brônquica, incluindo verificação de resposta angular, limites de curso, sentido de rotação, ruído, aquecimento e repetibilidade. A seguir, deve ser testado o manipulador delta sem carga, confirmando se a cinemática inversa produz movimentos coerentes nos eixos X , Y e Z .

Depois da montagem com o modelo, os ensaios devem verificar se os movimentos esperados são obtidos sem folgas excessivas nem bloqueios. As amplitudes de referência são 5 a 10 mm para respiração ao nível da traqueia, 1 a 3 mm para batimento cardíaco e valores superiores, de curta duração, para tosse. Se o deslocamento real for inferior ao pretendido, deve ser ajustada a amplitude dos pontos da trajetória ou a geometria de ligação.

5.4 Testes de alimentação

A alimentação deve ser validada em três condições, repouso, movimento nominal e situação de maior esforço, tendo em conta que, para os servos, foram estimados consumos nominais de cerca de 150 a 250 mA por unidade e correntes superiores em bloqueio. Assim, a fonte dos servos deve ser dimensionada para picos e não apenas para consumo médio.

O Raspberry Pi 4B e o ESP32-S3 devem manter alimentação estável durante a comunicação e durante o acionamento dos motores. A utilização de UPS/HAT deve ser testada abrindo e fechando a alimentação principal, confirmando se o sistema tem tempo para guardar estado, alertar ou encerrar de forma controlada.

5.5 Testes de software

Os testes de software devem começar pela inicialização das estruturas `MotionStorage` e `MotionInstance`, cujas funções de depuração devem confirmar o número de pontos, período, pausas, índices e ponteiros. Em seguida, a FSM de movimento deve ser testada com trajetórias simples, verificando se os estados evoluem de forma esperada e se a interpolação entre pontos é contínua.

A FSM de comandos deve ser testada com entradas série conhecidas, de forma a que cada comando produza apenas a alteração prevista: ativar respiração, ativar bati-

mento, ativar modo combinado, disparar tosse ou alterar parâmetros. Já os comandos com parâmetros numéricos devem ser testados com valores válidos, valores limite e entradas incompletas.

5.6 Testes de comunicação série

Durante o desenvolvimento foi identificado que a abertura da porta série por Python ou pelo monitor série pode provocar reset do ESP32, pelo que este efeito deve ser controlado durante os ensaios. Um teste simples consiste em imprimir um contador no `setup()` e observar se este é reiniciado quando a porta é aberta, o que permite distinguir falhas de firmware de resets causados pela interface USB/serial.

No lado Python, a leitura deve remover caracteres de fim de linha com `strip()` e evitar imprimir mensagens vazias. A comunicação deve ser validada em dois sentidos: mensagens enviadas pelo ESP32 para monitorização e comandos enviados pelo computador ou Raspberry Pi para selecionar modos.

5.7 Testes de integração dos movimentos

A integração deve ser feita por etapas: primeiro executa-se apenas respiração, verificando periodicidade, amplitude e suavidade; depois adiciona-se o batimento cardíaco, que deve aparecer como pequena oscilação sobreposta; por fim, testa-se a tosse, que deve alterar temporariamente o comportamento da respiração e ativar vibração ou perturbação curta.

O modo combinado é o mais importante para avaliar o realismo, pois aproxima o comportamento de um ambiente endoscópico dinâmico. Neste modo, devem ser observados conflitos entre movimentos, saturação dos servos, perda de suavidade e eventuais posições impossíveis da cinemática inversa.

5.8 Critérios de aceitação

Para esta fase do projeto, os critérios de aceitação podem ser definidos de forma funcional:

- o sistema arranca sem movimentos inesperados perigosos;
- os servos respondem dentro dos limites definidos;
- a respiração apresenta movimento cíclico, suave e repetível;
- o batimento acrescenta uma oscilação de pequena amplitude;
- a tosse produz um evento curto e abrupto, recuperando depois para o estado normal;

- os comandos série selecionam os modos sem bloquear o firmware;
- a alimentação mantém tensão estável durante os movimentos.

5.9 Informação experimental em falta

Para transformar este plano numa análise final de resultados, falta recolher dados quantitativos. Em particular, devem ser medidos deslocamentos reais nos eixos X , Y e Z , frequência efetiva dos movimentos, erro entre posição desejada e posição observada, consumo elétrico, aquecimento dos servos, estabilidade da fonte e tempo de resposta aos comandos série.

Também falta documentar a avaliação visual do movimento com o broncoscópio introduzido no modelo. Esta etapa é importante porque o realismo percebido não depende apenas da amplitude mecânica, mas também da forma como o movimento aparece na imagem endoscópica.

Foi realizada uma sessão preliminar de observação com um profissional de saúde, em que o broncoscópio foi introduzido no modelo impresso em 3D enquanto o sistema estava ativo (figura 5.1). Esta sessão permitiu obter uma primeira avaliação qualitativa da disposição dos ramos, do espaço disponível para a progressão da sonda e da perceção de movimento durante o funcionamento do manipulador. As observações desta sessão ajudaram a identificar aspetos a melhorar na geometria do modelo e no comportamento do sistema, mas os dados quantitativos ainda não foram recolhidos de forma sistemática.



Figura 5.1: Sessão de observação preliminar com profissional de saúde: broncoscópio introduzido no modelo de árvore brônquica impresso em 3D enquanto o sistema de controle estava ativo, junho de 2026.

Capítulo 6

Conclusão

6.1 Síntese do trabalho

Este projeto tem como objetivo desenvolver um modelo articulado de uma árvore brônquica para treino de videobroncofibroscopia, cuja solução proposta combina impressão 3D, atuação mecânica, controle por ESP32, comunicação com Raspberry Pi 4B, interface HMI e perfis de movimento inspirados em fenômenos fisiológicos.

O desenvolvimento realizado permitiu consolidar os requisitos do sistema, segundo os quais o modelo deve ser acessível, modular, fácil de montar e capaz de reproduzir movimentos relevantes durante uma broncoscopia flexível. Para esta versão, foram priorizados três fenômenos, respiração, batimento cardíaco e tosse, tendo outros efeitos, como deformação localizada do lúmen, estenose e resistência ao contacto com o broncoscópio, sido identificados como importantes mas reservados para trabalho futuro devido à maior complexidade mecânica.

6.2 Principais contributos

Os principais contributos do trabalho são:

- definição de requisitos para um simulador dinâmico de árvore brônquica;
- identificação dos movimentos fisiológicos mais relevantes para uma primeira versão funcional;
- escolha de uma arquitetura mecânica baseada em manipuladores delta;
- definição da arquitetura eletrónica com Raspberry Pi 4B, ESP32-S3, servos e sistema de alimentação;
- desenvolvimento de uma arquitetura de firmware baseada em estruturas de trajetória, instâncias de execução e máquinas de estados;

- desenvolvimento de uma HMI para seleção de modos, envio de comandos, monitorização da comunicação série e configuração de movimentos;
- integração lógica da tosse como evento temporário sobreposto ao movimento respiratório.

6.3 Limitações

O protótipo ainda apresenta limitações importantes, desde logo porque o realismo material é limitado pelos processos de impressão disponíveis e pelos filamentos acessíveis. A solução de movimento em três eixos simplifica a dinâmica real da árvore traqueobrônquica, que inclui deformações locais, variações de calibre e interação com o broncoscópio, e a validação quantitativa dos movimentos também depende de medições experimentais futuras, nomeadamente amplitudes reais, repetibilidade, ruído, aquecimento e estabilidade da alimentação.

Outra limitação é a ausência, nesta fase, de feedback sensorial: sem sensores de contacto, força ou posição externa, o sistema depende essencialmente do comando aberto dos servos e das validações geométricas implementadas no firmware. Isto é aceitável para uma primeira prova de conceito, mas limita a simulação de resistência ao avanço do broncoscópio.

6.4 Trabalho futuro

A linha mais importante de evolução diz respeito ao modelo físico da árvore brônquica. A versão atual não simula deformação ativa do lúmen, variação de calibre ao longo do ciclo respiratório, estenoses localizadas ou resistência ao avanço do broncoscópio. Estes fenómenos têm impacto direto na perceção visual e tátil durante o procedimento real, mas exigem materiais com propriedades mecânicas controladas, atuadores distribuídos ao longo da estrutura e uma estratégia de controlo mais complexa. A integração de filamentos com diferentes durezas em zonas específicas do modelo, combinada com pequenos atuadores de deformação local, constitui a principal direção técnica para uma segunda versão.

No plano da eletrónica e da segurança, a adição de sensores de fim de curso, medição de corrente dos servos e possível feedback de posição permitiria detetar sobrecargas e posições impossíveis, aumentar a robustez do sistema e abrir caminho para simulação de resistência ao contacto com as paredes internas. Estes elementos não foram integrados nesta versão por razões de complexidade e tempo de desenvolvimento, mas a arquitetura de firmware está organizada de forma a facilitar a sua adição.

A seleção de materiais flexíveis para a árvore brônquica deve ser aprofundada em versões futuras, comparando TPU, TPE, PEBA e outros filamentos quanto à facilidade

de impressão, elasticidade, durabilidade e sensação tátil durante a navegação endoscópica. Esta análise, que nesta versão ficou limitada a testes qualitativos, beneficiaria de ensaios mecânicos sistemáticos.

A interface gráfica no Raspberry Pi tem margem de evolução significativa: personalização de cenários, envio completo de trajetórias para o ESP32 por comunicação série, armazenamento de perfis de treino em ficheiro e monitorização mais detalhada do estado interno do firmware. A arquitetura Flask/SocketIO já utilizada facilita a adição de novas páginas e funcionalidades sem alterar a lógica de baixo nível.

Por fim, a validação com profissionais de saúde, iniciada de forma qualitativa em junho de 2026, deve ser aprofundada com uma metodologia estruturada. Esta avaliação é essencial para perceber se o movimento gerado tem valor pedagógico real, que aspetos do simulador são mais importantes para o treino de videobroncofibroscopia e que limitações da versão atual têm mais impacto na usabilidade clínica.

Bibliografia

- [1] Mohsen Davoudi e Henri G. Colt. «Bronchoscopy simulation: a brief review». Em: *Advances in Health Sciences Education* 14.2 (2008), pp. 287–296. DOI: [10.1007/s10459-007-9095-x](https://doi.org/10.1007/s10459-007-9095-x).
- [2] Cassie C. Kennedy, Fabien Maldonado e David A. Cook. «Simulation-Based Bronchoscopy Training: Systematic Review and Meta-analysis». Em: *Chest* 144.1 (2013), pp. 183–192. DOI: [10.1378/chest.12-1786](https://doi.org/10.1378/chest.12-1786).
- [3] David I. Fielding, Fabien Maldonado e Septimiu Murgu. «Achieving competency in bronchoscopy: Challenges and opportunities». Em: *Respirology* 19.4 (2014), pp. 472–482. DOI: [10.1111/resp.12279](https://doi.org/10.1111/resp.12279).
- [4] Philip Mørkeberg Nilsson et al. «Simulation in bronchoscopy: current and future perspectives». Em: *Advances in Medical Education and Practice* 8 (2017), pp. 755–760. DOI: [10.2147/AMEP.S139929](https://doi.org/10.2147/AMEP.S139929).
- [5] Mallika Gopal et al. «Bronchoscopy Simulation Training as a Tool in Medical School Education». Em: *The Annals of Thoracic Surgery* 106.1 (2018), pp. 280–286. DOI: [10.1016/j.athoracsur.2018.02.011](https://doi.org/10.1016/j.athoracsur.2018.02.011).
- [6] Pedro Barros, Bruno Santos e Ulisses Brito. «Projeto para desenvolvimento de modelo de simulação de árvore brônquica, com impressão 3D». Documento de trabalho interno, Serviço de Pneumologia, Centro Hospitalar Universitário do Algarve, cedido à equipa do projeto. 2023.
- [7] T. H. Pedersen et al. «A randomised, controlled trial evaluating a low cost, 3D-printed bronchoscopy simulator». Em: *Anaesthesia* 72.8 (2017), pp. 1005–1009. DOI: [10.1111/anae.13951](https://doi.org/10.1111/anae.13951).
- [8] Rao Fu et al. «Dynamic three-dimensional printing: The future of bronchoscopic simulation training?» Em: *Anaesthesia and Intensive Care* 51.4 (2023), pp. 274–280. DOI: [10.1177/0310057X231154015](https://doi.org/10.1177/0310057X231154015).
- [9] Peter Heinbecker. «A method for the demonstration of calibre changes in the bronchi in normal respiration». Em: *Journal of Clinical Investigation* 4.4 (1927), pp. 459–469. DOI: [10.1172/JCI100133](https://doi.org/10.1172/JCI100133).

- [10] Maxwell Ellis. «The mechanism of the rhythmic changes in the calibre of the bronchi during respiration». Em: *The Journal of Physiology* 87.3 (1936), pp. 298–309. DOI: [10.1113/jphysiol.1936.sp003408](https://doi.org/10.1113/jphysiol.1936.sp003408).
- [11] Alexander Chen et al. «The Effect of Respiratory Motion on Pulmonary Nodule Location During Electromagnetic Navigation Bronchoscopy». Em: *Chest* 147.5 (2015), pp. 1275–1281. DOI: [10.1378/chest.14-1425](https://doi.org/10.1378/chest.14-1425).
- [12] Chamindu C. Gunatilaka et al. «The effect of airway motion and breathing phase during imaging on CFD simulations of respiratory airflow». Em: *Computers in Biology and Medicine* 127 (2020), p. 104099. DOI: [10.1016/j.compbiomed.2020.104099](https://doi.org/10.1016/j.compbiomed.2020.104099).
- [13] Bruno Santos. «Simulação dinâmica da videobroncofibroscopia em modelo impresso 3D: introdução de movimento no modelo de árvore brônquica». Documento de trabalho interno, Serviço de Pneumologia, Centro Hospitalar Universitário do Algarve, cedido à equipa do projeto. 2026.
- [14] William Bailes. *Delta-Kinematics-Library*. Biblioteca Arduino de código aberto para cinemática direta e inversa de manipuladores delta. 2018. URL: <https://github.com/tinkersprojects/Delta-Kinematics-Library> (acedido em 03/07/2026).
- [15] Alex Nikanorov. *Delta robot kinematics*. Tutorial online, HyperTriangle. URL: <https://hypertriangle.com/~alex/delta-robot-tutorial/> (acedido em 03/07/2026).
- [16] isaac879. *Delta-Robot*. Repositório de código aberto; robô manipulador delta impresso em 3D controlado por Arduino Nano. 2019. URL: <https://github.com/isaac879/Delta-Robot> (acedido em 03/07/2026).
- [17] P. J. Zsombor-Murray. *Descriptive Geometric Kinematic Analysis of Clavel's "Delta" Robot*. McGill University, Centre for Intelligent Machines, 2004. URL: <https://cim.mcgill.ca/~paul/clavdelt.pdf> (acedido em 03/07/2026).
- [18] M. Hasanlu e M. Siavashi. «Optimum kinematic–dynamic performance of the reconfigurable delta robot through genetic algorithm optimization». Em: *Robotic Systems and Applications* 5.1 (2025). DOI: [10.21595/rsa.2025.24731](https://doi.org/10.21595/rsa.2025.24731).

Apêndice A

Anexos

A.1 Anexo A: Desenhos técnicos e modelo 3D

Incluir vistas do modelo, ficheiros CAD relevantes, parâmetros de impressão 3D e notas de montagem.

A.2 Anexo B: Lista de materiais

Incluir lista completa de peças, fornecedores, referências, quantidades e custos.

A.3 Anexo C: Datasheets

Incluir datasheets dos principais componentes eletrónicos, atuadores, drivers e fonte de alimentação.

A.4 Anexo D: Código e configurações

Incluir excertos relevantes de firmware, ficheiros de configuração, mapas de pinos e instruções de carregamento.

A.5 Anexo E: Protocolos de teste

Incluir grelhas de teste, formulários de avaliação, tabelas de resultados brutos e notas de calibração.